Linux Mastery Tutoring Manual

From Beginner to Advanced — Linux Mint & Ubuntu, Backend Web Development, Web3 Infrastructure and Job-Ready Linux Skills

Prepared for Samuel Amakundi Bako — TriTech Academy

October 2026 — Edition 1.0

Contents

How to use this manual 4

Level 1 — Foundations 6

Module 1 — Linux, distributions and your lab 7

Module 2 — The terminal and the shell 10

Module 3 — Files, directories and the filesystem 13

Module 4 — Viewing and editing text 16

Module 5 — Users, groups, permissions and sudo 18

Module 6 — Installing and maintaining software 22

Level 2 — Power user 25

Module 7 — Redirection, pipes and text processing 26

Module 8 — Processes, jobs and signals 30

Module 9 — systemd: services, logs, timers and boot 33

Module 10 — Storage, filesystems, archives and backups 37

Module 11 — Networking essentials 40

Module 12 — Bash scripting I: the language 43

Module 13 — Bash scripting II: production-grade automation 47

Level 3 — Backend engineer 51

Module 14 — The backend developer's workstation 52

Module 15 — Databases and caches on Linux 55

Module 16 — Deploying a backend: Gunicorn, systemd, Nginx and HTTPS 57

Module 17 — Containers with Docker 62

Module 18 — Security hardening 65

Module 19 — Monitoring, performance and troubleshooting 68

Module 20 — CI/CD and cloud servers 71

Level 4 — Web3 engineer 73

Module 21 — Web3 development toolchain on Linux 74

Module 22 — Running blockchain infrastructure on Linux 78

Capstone projects 82

Appendix A — Command cheat sheet 84

Appendix B — Quick references 85

Appendix C — Interview question bank..... 86

Appendix D — Checkpoint answer key (for tutors)..... 87

Appendix E — How the source documents were used..... 91

Appendix F — Suggested 20-week schedule..... 93

Appendix G — Glossary..... 94

How to use this manual

This manual teaches Linux from first login to running production backend and Web3 infrastructure. It is written for two readers at once:

- **The learner**, working through the lessons in order at a Linux terminal.
- **The tutor**, who uses the tutor notes, timings, checkpoints and assessment questions to run the sessions.

It is organised in four levels. Each level assumes the one before it.

Level	Modules	You will be able to…
1 — Foundations	1–6	Install and maintain Linux Mint or Ubuntu, move around the terminal, manage files, users, permissions and software
2 — Power user	7–13	Process text and logs, control processes and services, manage storage and networks, and write reliable Bash scripts
3 — Backend engineer	14–20	Build a Python/Node development environment, run databases, deploy a backend behind Nginx with TLS, use Docker, harden and monitor servers, and automate deployment
4 — Web3 engineer	21–22	Set up an EVM smart-contract toolchain (Foundry, Hardhat), handle keys safely, and run Ethereum and IPFS nodes as Linux services

The manual ends with four capstone projects, an interview question bank, cheat sheets and a map of how each source book was used.

Lesson format

Every lesson follows the same pattern:

- **Objectives** — what you can do at the end of the lesson.
- **Mental model** — a short analogy that makes the idea stick.
- **Walkthrough** — numbered steps with the exact commands. Type them; don't paste them in the early modules.
- **Expected output** — what you should see, so you can tell when something went wrong.
- **Try it** — short exercises that practise the lesson.
- **Common mistakes** — the errors learners actually make.

Each module ends with a **lab**, a **checkpoint quiz**, and **tutor notes**.

These callouts appear throughout:

> **Note:** extra context or a useful detail.
>
> **Warning:** a command or mistake that can lose data, lock you out or create a security hole.
>
> **Tutor note:** guidance for the person teaching the session: timings, demos and where learners usually get stuck.

Conventions

- `$` at the start of a line is the prompt of a normal user; `#` is the root prompt. Don't type them.
- Placeholders are written in angle brackets, e.g. `<username>`; replace them, including the brackets.
- Lines in `text` blocks are output, not commands.
- Commands were checked on Ubuntu 24.04 LTS with Bash 5.2. They apply unchanged to Linux Mint 22.x (built on Ubuntu 24.04) and to Ubuntu 26.04 LTS unless a difference is noted.

Which Linux to use

Use	Recommended system	Why
Your daily workstation	**Linux Mint 22.3 "Zena"** (Cinnamon edition)	Friendly desktop, Ubuntu 24.04 package base, very stable
Servers and cloud VMs	**Ubuntu Server 24.04 LTS or 26.04 LTS**	The most common cloud image; long-term support; what employers expect
Learning safely	A virtual machine (VirtualBox, GNOME Boxes or virt-manager)	Mistakes cost nothing; snapshots let you rewind

> **Note:** Ubuntu 26.04 LTS "Resolute Raccoon" was released in April 2026. Linux Mint's next release, **Mint 27 "Adrien"**, is planned for Christmas 2026 on the Ubuntu 26.04 base; Mint is moving to one release per year and dropping point releases. Everything in this manual works on both bases. Check your version with `cat /etc/os-release`.

A word on the source books

This manual draws on eight source documents:

- *Introduction to Linux — A Hands-on Guide* (two identical copies were supplied)
- *Bash Guide for Beginners*
- *Linux Command Line and Shell Scripting Bible*
- *The Linux System Administrator's Guide*
- *The Linux Network Administrator's Guide*
- your *Linux Mastery Guide* (DOCX and PDF)

It also draws on the official documentation: man pages, the GNU Bash manual, Ubuntu Server documentation, the Linux Mint user guide, systemd, Docker, Nginx, PostgreSQL, Foundry, Hardhat and Ethereum client docs.

Several source books were written between 2001 and 2014. Their concepts are still right, but many of their tools have been replaced:

Old tool	Modern replacement
`ifconfig`, `route`, `netstat`	`ip`, `ss`
SysV `init` and runlevels	systemd
`ipchains` / early `iptables`	nftables and `ufw`
LILO	GRUB 2
`inetd`	systemd socket units
`telnet` / `ftp`	SSH / SFTP

Wherever this manual teaches a topic the old books cover, it teaches the modern tool and mentions the legacy one only so you recognise it. Appendix E maps every source to the modules that use it.

Level 1 — Foundations

Level 1 turns a complete beginner into someone who can install Linux, find their way around the terminal, manage files and software, and understand who is allowed to do what on the system. Budget about four weeks at 6–8 hours a week.

Module 1 — Linux, distributions and your lab

Objectives: explain what Linux is and how a distribution differs from the kernel; choose between Mint and Ubuntu; build a safe practice lab; perform the first-boot routine.

1.1 What Linux actually is

Linux is a **kernel**: the core program that controls the CPU, memory, disks and network cards and shares them between running programs. Linus Torvalds started it in 1991. A complete operating system adds:

- the **GNU tools**, such as `ls`, `cp`, `grep` and the Bash shell;
- a **package manager**, which installs and updates software;
- **system services**, such as systemd and networking;
- usually a **desktop environment**.

A **distribution** ("distro") packages all of these together.

> **Mental model:** the kernel is the engine of a car. A distribution is the finished car, with seats, dashboard and warranty. Ubuntu and Mint use the same engine and much of the same body; Mint changes the dashboard (the Cinnamon desktop) and some defaults.

The system is built in layers:

Layer	Examples	What it does
Hardware	CPU, RAM, SSD, NIC	Physical resources
Kernel	Linux 6.x / 7.x	Scheduling, memory, drivers, filesystems, networking
System libraries	glibc, OpenSSL	Code that programs share
System services	systemd, NetworkManager, sshd	Start and supervise background programs
Shell and tools	Bash, coreutils, `apt`	Your command-line interface
Applications	Firefox, VS Code, PostgreSQL, Nginx	What you actually use

Why it matters for your career:

- The large majority of web servers and cloud virtual machines run Linux.
- Every Docker container is Linux.
- Ethereum clients and most blockchain nodes are developed and run primarily on Linux.

1.2 Distributions you will meet

Family	Distros	Package tool	Where you meet them
Debian	Debian, **Ubuntu**, **Linux Mint**, Pop!_OS	`apt`, `dpkg`	Laptops, most cloud VMs, Docker base images
Red Hat	RHEL, Rocky, AlmaLinux, Fedora	`dnf`, `rpm`	Enterprises, banks, telecoms
Arch	Arch, Manjaro	`pacman`	Enthusiasts
Alpine	Alpine	`apk`	Tiny Docker images
SUSE	openSUSE, SLES	`zypper`	Parts of Europe and SAP shops

Linux Mint is built on Ubuntu LTS: Mint 22.x uses Ubuntu 24.04 packages, and Mint 27 will use Ubuntu 26.04. Almost everything you learn on Mint therefore works on an Ubuntu server, which is exactly why this pairing is ideal for a backend developer.

Mint differs from Ubuntu in a few ways you should know:

Topic	Linux Mint	Ubuntu
Desktop	Cinnamon (also MATE, Xfce)	GNOME
Snap packages	Disabled by default (Flatpak preferred)	Built in
Updates	Update Manager with update levels and Timeshift integration	Software Updater / `unattended-upgrades`
System snapshots	Timeshift set up in the first-run wizard	Not set up by default
`/etc/os-release`	`ID=linuxmint`, `UBUNTU_CODENAME=noble`	`ID=ubuntu`

> **Note:** many third-party install guides (Docker, PostgreSQL's own repository, etc.) read `VERSION_CODENAME` to pick a package repository. On Mint that is Mint's own codename (e.g. `zena`), which those repositories do not know. Use `UBUNTU_CODENAME` instead. Module 6 shows how.

1.3 Build your lab

You need somewhere to make mistakes. Choose one option.

Option A — Virtual machine (recommended for weeks 1–4).

1. Install VirtualBox, or on an existing Linux machine use `virt-manager` or GNOME Boxes.
2. Download the Linux Mint 22.3 Cinnamon ISO from linuxmint.com and the Ubuntu Server 24.04 LTS ISO from ubuntu.com. Verify each download's checksum (step 5 below).
3. Create a VM with 2 vCPUs, 4 GB RAM (8 GB for Docker and Web3 work later) and a 40 GB disk.
4. Install, then **take a snapshot** called `fresh-install`.
5. Verify downloads before installing. In the folder containing the ISO and the published `sha256sum.txt`:

```
sha256sum -c sha256sum.txt --ignore-missing
linuxmint-22.3-cinnamon-64bit.iso: OK
```

Option B — Linux Mint as your main OS. Install on the whole disk, or dual-boot with Windows. Before you start:

- Back up everything.
- Shrink the Windows partition from Windows Disk Management, not from the Linux installer.
- Disable Windows Fast Startup.

> **Warning:** the installer's "Erase disk" option means exactly that. Read every partitioning screen twice.

Option C — WSL 2 on Windows. Run `wsl --install -d Ubuntu-24.04` in an administrator PowerShell. This is good for command-line practice, but systemd, networking and disks behave differently, so do Modules 9–11 in a VM.

Option D — Cloud VM. A small Ubuntu VM from any cloud provider gives you a real server with a public IP. You need this from Module 16 onwards.

1.4 First-boot routine (Mint and Ubuntu)

Do this on every new machine.

1. **Update the package lists and install updates.**

```
sudo apt update
sudo apt full-upgrade
```

`apt update` refreshes the catalogue of available packages; `apt full-upgrade` installs the newer versions, including ones that need new dependencies. On Mint you can also use **Update Manager**.

2. **Install drivers if needed.** On Mint, open **Driver Manager**; on Ubuntu run `ubuntu-drivers devices`.

3. **Set up Timeshift (Mint).** Choose **RSYNC** mode, select a disk with enough space, and keep 2 daily and 2 weekly snapshots. Timeshift protects system files, not your home folder; Module 10 covers backing up your data.

4. **Enable the firewall.**

```
sudo ufw enable
sudo ufw status verbose

Status: active
Default: deny (incoming), allow (outgoing), disabled (routed)
```

5. **Install basic tools you will use throughout the manual.**

```
sudo apt install -y build-essential git curl wget vim htop tree jq unzip ca-certificates
```

6. **Record your system details.**

```
cat /etc/os-release
uname -r
lscpu | head -15
free -h
df -h /
```

Lab 1 — Know your machine

Create a file `~/lab1.txt` that answers the questions below. Use only the terminal; how to write files is explained in Module 3, but `command >> ~/lab1.txt` appends a command's output.

1. Distribution name and version.
2. Kernel version.
3. Number of CPU cores and total RAM.
4. Free space on `/`.
5. Whether the firewall is active.

Checkpoint 1

1. What is the difference between the Linux kernel and a distribution?
2. Why is Linux Mint a good desktop for someone who deploys to Ubuntu servers?
3. Which `/etc/os-release` field should installation scripts use on Mint to find the Ubuntu base, and why?
4. What does `sudo apt update` do that `sudo apt full-upgrade` does not?
5. What does Timeshift protect, and what does it not protect?

> **Tutor note:** 3–4 hours. Spend the session building the VM together; nothing motivates like a working desktop. Insist on the snapshot before Module 2. Common blocker: virtualisation (VT-x/AMD-V) disabled in the BIOS/UEFI.

Module 2 — The terminal and the shell

Objectives: open a terminal, read the prompt, understand how a command is structured, get help without Google, and work fast with history and keyboard shortcuts.

2.1 Terminal, shell and prompt

- The **terminal** is the window (on Mint: GNOME Terminal; `Ctrl+Alt+T` opens it).
- The **shell** is the program inside it that reads your commands. Linux Mint and Ubuntu use **Bash** (the "Bourne Again SHell").
- The **prompt** tells you who and where you are:

```
amakundi@mint-lab:~/projects$
```

Part	Meaning
`amakundi`	Your username
`mint-lab`	The computer's hostname
`~/projects`	Current directory (~ = your home, `/home/amakundi`)
`$`	Normal user (# means root, the administrator)

> **Mental model:** the shell is a very literal assistant. It does exactly what you type, immediately, without "Are you sure?" dialogs. That is its power and its danger.

2.2 Anatomy of a command

```
command  -options  arguments
ls       -l -h     /var/log
```

- **Options** change behaviour. Short options start with one dash and can be combined (`-lh`); long options start with two dashes (`--human-readable`).
- **Arguments** say what to act on.
- Spaces separate the parts, which is why file names containing spaces need quotes (Module 3).

Walkthrough — type each command and observe:

```
date
cal
whoami
hostname
echo "Hello, Linux"
ls
ls -l
ls -lh /var/log
uptime
```

2.3 Getting help (the most important skill in this manual)

Command	When to use it
`command --help`	Quick summary of options
`man command`	Full manual page. Navigate with arrows, `/word` to search, `n` for the next match, `q` to quit
`man -k keyword` (apropos)	"Which command does X?"
`info command`	Longer GNU manuals (e.g. `info coreutils`)
`type command`	Is it a builtin, an alias, a function or a program?
`help builtin`	Help for shell builtins such as `cd`, `read` and `for`
`tldr command`	Community examples. Install with `sudo apt install`

Command	When to use it
	`tldr`, then run `tldr --update`

```
man ls
man -k "copy files"
type cd ls ll
help cd

cd is a shell builtin
ls is aliased to `ls --color=auto'
ll is aliased to `ls -alF'
```

Man pages are organised in **sections**:

- 1: user commands
- 5: file formats
- 8: administration commands

`man 5 passwd` shows the file format; `man 1 passwd` shows the command.

2.4 Working fast: history and shortcuts

Keys / command	Action
`Tab` / `Tab Tab`	Complete a name / show the possibilities
`↑` `↓`	Previous / next command
`Ctrl+R`	Search history backwards (press again for older matches)
`Ctrl+A` / `Ctrl+E`	Jump to the start / end of the line
`Ctrl+W` / `Ctrl+U`	Delete the previous word / everything before the cursor
`Ctrl+C`	Cancel the running command
`Ctrl+D`	End of input; at an empty prompt it closes the shell
`Ctrl+L` or `clear`	Clear the screen
`history`, `!42`, `!!`, `!$`	Show history, rerun entry 42, rerun the last command, last argument of the previous command

A classic use of `!!`:

```
apt install htop        # fails: permission denied
sudo !!                 # reruns it as: sudo apt install htop
```

2.5 Customising your shell

Bash reads `~/.bashrc` every time you open an interactive terminal. Add aliases at the end with `nano ~/.bashrc` (Module 4 teaches `nano`):

```
alias ll='ls -alFh'
alias update='sudo apt update && sudo apt full-upgrade'
alias ports='ss -tulpn'
export HISTSIZE=10000 HISTFILESIZE=20000
export HISTTIMEFORMAT='%F %T  '
```

Reload it without reopening the terminal:

```
source ~/.bashrc
```

Try it

1. Find the command that reports disk usage of a directory, using only `man -k`.
2. Show the last 20 commands with timestamps.
3. Use `Ctrl+R` to find and rerun your earlier `ls -lh /var/log`.

Common mistakes

- Typing the `$` from the book.

- Expecting `Ctrl+C/Ctrl+V` to copy and paste. In the terminal they are `Ctrl+Shift+C` and `Ctrl+Shift+V`.
- Getting stuck in `man` or `less`: press `q`.

Checkpoint 2

1. What does # at the end of the prompt mean?
2. Explain the difference between `ls -l -h` and `ls --human-readable -l`.
3. How do you find a command when you only know what it should do?
4. What file do you edit to make an alias permanent, and how do you apply it immediately?

> **Tutor note:** 3 hours. Make learners answer every question in this session using `man` before you help. The habit of reading man pages is what separates job-ready engineers from people who copy commands.

Module 3 — Files, directories and the filesystem

Objectives: understand the Linux directory tree, use absolute and relative paths, create, copy, move and delete safely, use wildcards, find files, and understand links.

3.1 One tree, no drive letters

Everything hangs off a single root, `/`. Disks, USB sticks and network shares are **mounted** into the tree. The layout follows the Filesystem Hierarchy Standard (FHS):

Path	Contents	You will use it for
`/`	Root of everything	—
`/home/<user>`	Your files (~)	Projects, dotfiles
`/root`	The root user's home	Rarely
`/etc`	System configuration (text files)	Nginx, SSH, systemd, users
`/var`	Variable data: `/var/log`, `/var/lib`, `/var/www`	Logs, database files, websites
`/usr`	Installed programs (`/usr/bin`) and libraries	Where packages put software
`/usr/local`, `/opt`	Software you install yourself	Manual installs
`/tmp`	Temporary files (cleared on reboot)	Scratch work
`/dev`	Device files (`/dev/sda`, `/dev/null`)	Disks, special files
`/proc`, `/sys`	Virtual files exposing kernel and process state	Monitoring, tuning
`/boot`	Kernel and boot loader files	Kernel updates
`/mnt`, `/media`	Mount points	USB drives, extra disks
`/srv`	Data served by this system	Shared project directories

> **Mental model:** `/etc` is the settings room, `/var` is the filing cabinet that keeps growing, `/usr` is the library of installed software, and `/home` is your personal office.

3.2 Paths

- An **absolute path** starts at `/`: `/home/amakundi/projects/api`.
- A **relative path** starts where you are: `projects/api`, `../notes`.
- Special names: `.` (here), `..` (parent), `~` (your home), `-` (with `cd`, the previous directory).

```
pwd
cd /var/log
pwd
cd ..
cd ~/Documents     # absolute, using the ~ shortcut
cd -               # back to the previous directory
cd                 # with no argument: go home
```

3.3 Listing

```
ls                # names
ls -l             # long format
ls -a             # include hidden "dot" files
ls -lh            # human-readable sizes
ls -lt            # newest first
ls -lS            # largest first
ls -R projects    # recursive
tree -L 2 ~       # tree view, 2 levels deep
```

Reading `ls -l`:

```
-rw-r--r-- 1 amakundi amakundi  1520 Oct 16 09:12 notes.md
drwxr-xr-x 3 amakundi amakundi  4096 Oct 16 09:10 projects
lrwxrwxrwx 1 root     root        21 Oct  1 08:00 vimrc -> /etc/vim/vimrc
```

The columns are: type and permissions, link count, owner, group, size in bytes, modification time, and name. The first character is the file type: `-` regular file, `d` directory, `l` symbolic link. Module 5 explains the permission letters.

3.4 Creating, copying, moving and deleting

Walkthrough — build a practice project:

```
mkdir -p ~/practice/api/{src,tests,docs,config}
cd ~/practice/api
touch README.md src/app.py config/{dev,prod}.env
tree

.
├── README.md
├── config
│   ├── dev.env
│   └── prod.env
├── docs
├── src
│   └── app.py
└── tests

5 directories, 4 files
```

The order of entries can differ slightly with your language settings. `mkdir -p` creates parent directories as needed. `{a,b}` is **brace expansion**: the shell rewrites `config/{dev,prod}.env` into `config/dev.env config/prod.env` before running the command.

```
cp README.md docs/                  # copy a file into a directory
cp -r src src_backup                # copy a directory (recursive)
cp -a src /tmp/src_copy             # archive: keep permissions, times, links
mv docs/README.md docs/GUIDE.md     # rename
mv src_backup /tmp/                 # move
rm /tmp/src_copy/app.py             # delete a file
rmdir tests                         # delete an EMPTY directory
rm -r /tmp/src_backup               # delete a directory and its contents
```

Safety options worth knowing:

```
cp -i a b      # ask before overwriting
mv -n a b      # never overwrite
rm -I *        # ask once before deleting more than three files
```

> **Warning:** `rm` does not use the Trash. `rm -rf` with a wrong path is the most famous way to destroy a system. Before running a destructive command with a wildcard, run `ls` with the same pattern to see exactly what it matches.

3.5 Wildcards (globbing)

The shell expands wildcards before the command runs:

Pattern	Matches
`*`	Any characters (`*.py` = every Python file here)
`?`	Exactly one character (`log?.txt`)
`[abc]`, `[0-9]`	One character from a set or range
`**`	Any depth, after `shopt -s globstar`

```
ls *.env
ls config/*.env
echo src/*.py          # echo shows what the shell expanded
```

3.6 Names with spaces and special characters

```
mkdir "My Notes"        # quotes keep it one name
cd My\ Notes            # or escape the space
cd ..
```

Tip: avoid spaces in file names you create; use `my-notes` or `my_notes`.

3.7 Finding files

```
find ~/practice -name "*.env"                 # by name
find /var/log -type f -mtime -1               # files modified in the last day
find ~ -type f -size +100M                    # files bigger than 100 MB
find . -name "__pycache__" -type d -prune -exec rm -rf {} +   # remove Python caches
locate nginx.conf                             # fast database search (sudo apt install plocate)
which python3                                 # which program runs for a command
```

`find` walks the tree live. `locate` searches an index that is refreshed daily (`sudo updatedb` refreshes it now).

3.8 Links

- A **hard link** is a second name for the same file data. It cannot cross filesystems or point to directories.
- A **symbolic (soft) link** is a small file that points to a path. It is used everywhere in `/etc`, for example Nginx `sites-enabled` (Module 16).

```
echo "v1" > config/current.txt
ln -s current.txt config/active      # symlink: active -> current.txt
ls -l config/active
cat config/active

lrwxrwxrwx 1 amakundi amakundi 11 Oct 16 09:30 config/active -> current.txt
v1
```

3.9 File types and sizes

```
file /bin/ls README.md
stat README.md
du -sh ~/practice          # size of a directory tree
df -h                      # free space per mounted filesystem
```

Lab 3 — Organise a messy download folder

1. Create `~/lab3/downloads` and fill it with test files:

```
mkdir -p ~/lab3/downloads && cd ~/lab3/downloads
touch report{1..3}.pdf photo{1..4}.jpg notes.txt setup.deb "old file.zip"
```

2. Create `pdf/`, `images/` and `other/`, and move each file type into the right folder using wildcards.
3. Use `find ~/lab3 -type f | wc -l` to confirm that no files were lost (there should be 10).
4. Create a symlink `~/lab3/latest-report` pointing to `report3.pdf`.

Checkpoint 3

1. Which directory holds system-wide configuration, and which holds logs?
2. You are in `/home/amakundi/practice/api/src`. Write the relative path to `/home/amakundi/practice/api/config/dev.env`.
3. What is the difference between `rm -r` and `rmdir`?
4. When does the shell expand `*.py`: before or after the command runs? Why does that matter?
5. Give two differences between hard and symbolic links.

> **Tutor note:** 4 hours. The source books (*Introduction to Linux*, chapter 3; *Linux Command Line and Shell Scripting Bible*, chapters 3–4) cover this in depth. Learners always confuse relative paths; draw the tree on a whiteboard and walk it with `cd`.

Module 4 — Viewing and editing text

Objectives: read files of any size, follow logs live, and edit configuration files confidently with `nano` and with enough `vim` to survive on any server.

4.1 Reading files

Command	Use
`cat file`	Print a short file
`cat -n file`	With line numbers
`less file`	Page through a long file. `Space`/`b` = page down/up, `/word` = search, `n` = next match, `G` = end, `g` = start, `F` = follow, `q` = quit
`head -n 20 file`	First 20 lines
`tail -n 50 file`	Last 50 lines
`tail -f file`	Follow new lines as they are written (`Ctrl+C` to stop)
`tail -F file`	Follow by name, surviving log rotation
`wc -l file`	Count lines (`-w` words, `-c` bytes)

```
less /etc/services
head -n 5 /etc/passwd
sudo tail -f /var/log/syslog        # Mint/Ubuntu system log; Ctrl+C to stop
```

On systems without `/var/log/syslog` (minimal server images), use `journalctl -f` instead (Module 9).

4.2 nano — the friendly editor

```
nano ~/practice/api/config/dev.env
```

The shortcut bar at the bottom uses `^` for `Ctrl` and `M-` for `Alt`:

Keys	Action
`Ctrl+O`, `Enter`	Save
`Ctrl+X`	Exit
`Ctrl+W`	Search
`Ctrl+\`	Search and replace
`Ctrl+K` / `Ctrl+U`	Cut line / paste
`Alt+U`	Undo
`Ctrl+_`	Go to a line number

Editing a system file needs root:

```
sudo nano /etc/hosts
```

4.3 vim survival kit

`vim` (or `vi`) is installed on almost every server, including minimal images where `nano` may be missing. You need at least this much.

> **Mental model:** `vim` has *modes*. **Normal** mode is for moving and commanding; **Insert** mode is for typing text. `Esc` always takes you back to Normal mode.

In Normal mode	Action
`i`	Insert before the cursor (`a` after, `o` new line below)

In Normal mode	Action
`Esc`	Back to Normal mode
`:w`	Save
`:q`	Quit (`:q!` quit and discard changes)
`:wq` or `ZZ`	Save and quit
`h j k l`	Left, down, up, right
`gg` / `G`	Top / bottom
`/text`, `n`	Search, next match
`dd`, `yy`, `p`	Delete line, copy line, paste
`u`, `Ctrl+R`	Undo, redo
`:%s/old/new/g`	Replace everywhere
`:set number`	Show line numbers

Practice with `vimtutor`, an interactive 30-minute lesson that is installed with `vim`.

4.4 Graphical and IDE editors

On Mint, **Xed** (Text Editor) is the default graphical editor. For development, install Visual Studio Code from Microsoft's `.deb` package or apt repository (Module 14), or VSCodium from Flatpak. On servers, prefer editing over SSH with VS Code's **Remote - SSH** extension rather than copying files back and forth.

4.5 Comparing files

```
diff -u config/dev.env config/prod.env      # unified diff, the format Git uses
cmp file1 file2                             # byte comparison
sha256sum file                              # fingerprint to compare files across machines
```

Lab 4 — Configuration editing drill

1. Copy `/etc/ssh/ssh_config` to `~/lab4/` (no root needed to read it).
2. Using `vim` only, turn on line numbers, find the line containing `ForwardAgent`, uncomment it and change its value, then save.
3. Using `nano`, add a comment line at the top, then save.
4. Show your changes with `diff -u /etc/ssh/ssh_config ~/lab4/ssh_config`.

Checkpoint 4

1. Which command follows a log file even after it has been rotated?
2. In `vim`, how do you quit without saving?
3. Why should you know `vim` even if you prefer `nano` or VS Code?

> **Tutor note:** 2 hours. Run `vimtutor` in class for 30 minutes. Learners panic when trapped in `vim`; make them escape (`Esc :q!`) five times on purpose.

Module 5 — Users, groups, permissions and sudo

Objectives: read and change permissions in both symbolic and numeric form, manage users and groups, use `sudo` correctly, and understand the special bits used on shared and service directories.

5.1 Users and groups

Every process runs as a **user** and every file is owned by a **user** and a **group**.

- **root** (UID 0) can do anything.
- **System users** (UID < 1000) run services: `www-data` runs Nginx, `postgres` runs PostgreSQL.
- **Regular users** have a UID of 1000 or more.

```
id
whoami
groups
getent passwd "$USER"
```

```
uid=1000(amakundi) gid=1000(amakundi) groups=1000(amakundi),4(adm),27(sudo),100(users)
```

Membership in `sudo` is what lets you run admin commands on Mint and Ubuntu. On Red Hat systems the equivalent group is `wheel`.

The account databases:

File	Content (fields separated by `:`)
`/etc/passwd`	`name:x:UID:GID:comment:home:shell` (readable by everyone)
`/etc/shadow`	Password hashes and ageing information (root only)
`/etc/group`	`group:x:GID:member1,member2`

5.2 Managing accounts

Debian and Ubuntu provide friendly wrappers: `adduser` and `deluser`. The low-level tools `useradd`, `usermod` and `userdel` exist everywhere.

```
sudo adduser alice                      # interactive: creates home, sets password
sudo useradd -m -s /bin/bash bob        # non-interactive equivalent
sudo passwd bob
sudo groupadd devteam
sudo usermod -aG devteam alice          # -a appends; without -a you REPLACE all groups
sudo gpasswd -d alice devteam           # remove from a group
sudo chage -l alice                     # password ageing
sudo deluser --remove-home bob
```

Service accounts — use these for applications (Module 16):

```
sudo adduser --system --group --home /srv/myapi --shell /usr/sbin/nologin myapi
```

> **Warning:** `usermod -G group user` without `-a` removes the user from every other group, including `sudo`. Always use `-aG`.

Group changes take effect at your next login. In the current shell, `newgrp devteam` applies one immediately.

5.3 Reading permissions

```
-rwxr-x---  1 alice devteam  812 Oct 16 10:00 deploy.sh
 └┬┘└┬┘└┬┘
 user group others
```

Letter	On a file	On a directory
`r` (4)	Read the content	List the names inside
`w` (2)	Change the content	Create, delete and rename entries

Letter	On a file	On a directory
		inside
`x` (1)	Execute it as a program	Enter it (`cd`) and reach the files inside

So `rwxr-x---` means:

- the owner (alice) can read, write and execute;
- members of `devteam` can read and execute;
- everyone else can do nothing.

> **Mental model:** a directory is a room. `r` lets you see the list of names on the door, `x` lets you walk in, and `w` lets you move furniture in and out. Deleting a file needs `w` on the *directory*, not on the file.

5.4 chmod — changing permissions

Numeric (octal) form: add r=4, w=2, x=1 for each of user, group and others.

Octal	Symbolic	Typical use
600	`rw-------`	Private keys, `.env` files with secrets
644	`rw-r--r--`	Normal files, web content
640	`rw-r-----`	Config readable by a service's group
700	`rwx------`	Private directories, `~/.ssh`
755	`rwxr-xr-x`	Scripts, program directories, web roots
750	`rwxr-x---`	Application directories
775	`rwxrwxr-x`	Shared team directories

```
cd ~/practice/api
touch deploy.sh
chmod 755 deploy.sh
chmod 600 config/prod.env
chmod u+x,g-w,o= deploy.sh      # symbolic: add x for user, remove w from group, clear others
chmod -R g+rwX shared/          # capital X: execute only for directories (and files already
executable)
ls -l deploy.sh config/prod.env

-rwxr-x--- 1 amakundi amakundi 0 Oct 16 10:05 deploy.sh
-rw------- 1 amakundi amakundi 0 Oct 16 10:05 config/prod.env
```

5.5 chown and chgrp

```
sudo chown alice deploy.sh
sudo chown alice:devteam deploy.sh
sudo chown -R www-data:www-data /var/www/site
sudo chgrp devteam deploy.sh
```

> **Warning:** never "fix" a permission problem with `chmod -R 777`. It makes files writable by every process on the machine, including a compromised web server. Find out which user needs which access and grant exactly that.

5.6 umask — default permissions

New files start at 666 and new directories at 777, minus the **umask**:

```
umask           # 0002 for normal users on Mint/Ubuntu (each user has a private group); 0022 for
root
umask 027       # new files 640, new directories 750, for this shell only
```

5.7 Special bits

Bit	Octal	Effect	Example
setuid	4000	A program runs as its owner	`/usr/bin/passwd` (`-rwsr-xr-x`)
setgid on a directory	2000	New files inherit the directory's group	Shared team folders
sticky on a directory	1000	Only a file's owner can delete it	`/tmp` (`drwxrwxrwt`)

Shared team directory walkthrough:

```
sudo groupadd devteam
sudo usermod -aG devteam "$USER"
sudo mkdir -p /srv/shared
sudo chown root:devteam /srv/shared
sudo chmod 2775 /srv/shared
ls -ld /srv/shared

drwxrwsr-x 2 root devteam 4096 Oct 16 10:20 /srv/shared
```

The `s` in the group triplet is setgid. Any file created inside belongs to `devteam`, whoever creates it.

5.8 ACLs: permissions for one extra user

When owner/group/others is not enough, use Access Control Lists:

```
sudo setfacl -m u:alice:rx /srv/reports      # give alice read and execute
getfacl /srv/reports
```

A `+` at the end of the `ls -l` permission string shows that an ACL is set.

5.9 sudo and su

- `sudo command` runs one command as root and logs it. You type **your own** password.
- `sudo -i` opens a root login shell; leave it with `exit`. Use it sparingly.
- `su - alice` switches user and needs **alice's** password; `sudo -iu alice` does the same using your sudo rights.
- Configuration lives in `/etc/sudoers` plus files in `/etc/sudoers.d/`. **Always edit with `visudo`**, which checks the syntax before saving, because a broken sudoers file can lock you out of root.

```
sudo visudo -f /etc/sudoers.d/deploy

# let the deploy user restart one service without a password
deploy ALL=(root) NOPASSWD: /usr/bin/systemctl restart myapi
```

Log of sudo use: `sudo journalctl _COMM=sudo` or `/var/log/auth.log`.

> **Note:** Ubuntu 26.04 uses `sudo-rs`, a memory-safe reimplementation of sudo, by default. Its commands and the sudoers format are the same for everything in this manual.

Lab 5 — Secure shared workspace

1. Create users `alice` and `bob` and a group `webdev` containing both.
2. Create `/srv/webapp` owned by `root:webdev` with permissions so that:
   - group members can create files, and those files belong to `webdev`;
   - other users cannot list it.
3. As `alice` (`sudo -iu alice`), create `notes.txt`. As `bob`, confirm you can edit it.
4. Create `/srv/webapp/secret.env` readable only by `alice`.
5. Allow `bob` to run `systemctl status nginx` via `sudo` without a password, using a file in `/etc/sudoers.d/`.
6. Write down the octal permissions of everything you created, and justify each.

Checkpoint 5

1. What permissions does `chmod 640` give, and when would you use it?
2. Why do you need `w` on a directory to delete a file inside it?
3. What does the `s` in `drwxrwsr-x` do?
4. What is the danger of `usermod -G` without `-a`?
5. Why must you edit sudoers with `visudo`?
6. Why should a web application never run as root?

> **Tutor note:** 4 hours. Permissions cause more real-world tickets than any other Linux topic. Make learners predict the result of every `chmod` before running it. Sources: *Introduction to Linux*, chapters 3 and 7; *Linux Command Line and Shell Scripting Bible*, chapter 7; *The Linux System Administrator's Guide*, chapter 11.

Module 6 — Installing and maintaining software

Objectives: install, update and remove software the Debian/Ubuntu way; understand repositories and signing keys; use Flatpak and Snap sensibly; keep a system patched; recover from a bad update.

6.1 How packages work

A **package** (`.deb`) contains a program plus metadata: version, dependencies and install scripts. **Repositories** are signed servers that publish packages. **APT** resolves dependencies and downloads packages; **dpkg** installs individual `.deb` files.

> **Mental model:** APT is an app store with a catalogue (`apt update` refreshes the catalogue). Repositories are the shops. Signing keys are each shop's seal, which proves the package wasn't tampered with.

6.2 Everyday APT

```
sudo apt update                       # refresh package lists
apt list --upgradable                 # what would be upgraded
sudo apt upgrade                      # upgrade without removing packages
sudo apt full-upgrade                 # upgrade, allowing dependency changes
apt search postgresql                 # search
apt show nginx                        # details: version, size, dependencies
sudo apt install nginx postgresql     # install
sudo apt install ./package.deb        # install a local .deb, resolving dependencies
sudo apt remove nginx                 # remove, keep config files
sudo apt purge nginx                  # remove including system config
sudo apt autoremove                   # remove dependencies no longer needed
apt policy nginx                      # installed and candidate versions, and which repository
dpkg -l | grep nginx                  # installed packages
dpkg -L nginx                         # files installed by a package
dpkg -S /usr/sbin/nginx               # which package owns a file
sudo apt-mark hold linux-image-generic   # pin a package at its current version
```

History is kept in `/var/log/apt/history.log`, which is useful when you need to know what changed before something broke.

6.3 Where repositories are configured

System	Location
Ubuntu 24.04 / 26.04	`/etc/apt/sources.list.d/ubuntu.sources` (deb822 format)
Linux Mint 22.x	`/etc/apt/sources.list.d/official-package-repositories.list`, managed from **Software Sources**
Third-party repositories	Their own file in `/etc/apt/sources.list.d/`, with a key in `/etc/apt/keyrings/`

The modern, safe pattern for adding a third-party repository (Docker shown; the same shape works for most vendors). It works on both Ubuntu and Mint because it uses `UBUNTU_CODENAME`:

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
. /etc/os-release
sudo tee /etc/apt/sources.list.d/docker.sources > /dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: ${UBUNTU_CODENAME:-$VERSION_CODENAME}
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt update
```

> **Warning:** the old `apt-key add` method trusts a key for **every** repository and is deprecated. Always scope a key with `Signed-By`.

PPAs (`sudo add-apt-repository ppa:<owner>/<name>`) are personal repositories on Launchpad. Use only well-known ones; each one is code from a stranger running as root.

6.4 Flatpak and Snap

Format	Default on	Strength	Caveat
`.deb` via APT	Both	Integrated, small, system-wide	Versions can lag on LTS releases
Flatpak (Flathub)	**Mint** (in Software Manager)	Recent desktop apps, sandboxed	Larger; desktop apps only
Snap	**Ubuntu**	Recent apps, auto-updates, also servers	Disabled on Mint by default

```
flatpak search vscodium
flatpak install flathub com.vscodium.codium
flatpak update
snap list            # Ubuntu
sudo snap install --classic certbot
```

On Mint, Snap is blocked by `/etc/apt/preferences.d/nosnap.pref`. Remove that file only if you deliberately want Snap.

6.5 Building from source (when nothing else exists)

```
sudo apt install build-essential
./configure --prefix=/usr/local
make -j"$(nproc)"
sudo make install
```

Prefer packages. Software installed from source is not updated by `apt`, so record what you installed in `/usr/local` or `/opt`.

6.6 Keeping systems patched

Desktop (Mint). Update Manager → Edit → Preferences → Automation can apply updates automatically and remove obsolete kernels. Command-line equivalent:

```
mintupdate-cli upgrade
```

Server (Ubuntu). Install security updates automatically:

```
sudo apt install unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
cat /etc/apt/apt.conf.d/20auto-upgrades

APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
```

Check whether a reboot is needed:

```
[ -f /var/run/reboot-required ] && cat /var/run/reboot-required
```

Release upgrades.

- Ubuntu: `sudo do-release-upgrade`. LTS-to-LTS upgrades open after the first point release (e.g. 26.04.1).
- Mint: follow the official upgrade guide for each version.
- Always take a Timeshift snapshot or VM snapshot first.

6.7 Recovering from a bad update

1. Boot an older kernel: hold `Shift` (BIOS) or press `Esc` (UEFI) at boot to open GRUB, then choose **Advanced options**.
2. On Mint, restore the latest Timeshift snapshot (it also works from the live USB).

3. Fix interrupted installs:

```
sudo dpkg --configure -a
sudo apt -f install
```

4. Downgrade a package: `sudo apt install <package>=<version>`, using a version listed by `apt policy`.

Lab 6 — Workstation provisioning

1. Write the commands that install your standard toolset: `git`, `curl`, `jq`, `htop`, `tree`, `build-essential`, `python3-venv` and `tldr`.
2. Add the Docker repository using the scoped-key method (don't install Docker yet; that is Module 17).
3. Find which package provides `/usr/bin/ss` and list three other files it installs.
4. Enable automatic security updates on your Ubuntu Server VM and show the configuration.
5. Take a Timeshift snapshot on Mint and list snapshots with `sudo timeshift --list`.

Checkpoint 6

1. What is the difference between `apt remove` and `apt purge`?
2. Why is `Signed-By` safer than `apt-key add`?
3. Why do third-party install scripts sometimes fail on Mint, and how do you fix them?
4. When would you choose Flatpak over a `.deb`?
5. List the steps to recover after an update breaks the desktop.

> **Tutor note:** 3 hours. Sources: *Linux Command Line and Shell Scripting Bible*, chapter 9 (package management), updated for deb822 sources and keyrings; Ubuntu Server documentation (package management, automatic updates); the Linux Mint user guide.

Level 2 — Power user

Level 2 is where Linux starts paying for itself. You will process logs and data, control what runs on the machine, manage disks and networks, and automate work with Bash. Budget about six weeks.

Module 7 — Redirection, pipes and text processing

Objectives: connect commands with pipes; redirect output and errors; search with regular expressions; transform text with `grep`, `sed`, `awk`, `cut`, `sort`, `uniq`, `tr`, `xargs`; query JSON with `jq`.

7.1 The three standard streams

Every process starts with three open streams:

Stream	Number	Default
stdin (standard input)	0	Keyboard
stdout (standard output)	1	Terminal
stderr (standard error)	2	Terminal

> **Mental model:** each command is a machine on a factory line with one input chute and two output chutes: products (stdout) and rejects (stderr). Redirection reroutes the chutes; a pipe bolts one machine's product chute to the next machine's input.

Syntax	Meaning
`cmd > file`	stdout to a file (overwrite)
`cmd >> file`	stdout to a file (append)
`cmd 2> err.log`	stderr to a file
`cmd > all.log 2>&1`	stdout and stderr to the same file (order matters)
`cmd &> all.log`	Bash shorthand for the line above
`cmd > /dev/null 2>&1`	Discard everything
`cmd < file`	Read stdin from a file
`cmd1 | cmd2`	Pipe: stdout of cmd1 becomes stdin of cmd2
`cmd | tee file`	Show on screen **and** save to a file
`cmd <<'EOF' … EOF`	Here-document: multi-line input

```
ls /etc /nonexistent > out.txt 2> err.txt
cat out.txt | head -3
cat err.txt
find / -name "*.conf" 2>/dev/null | wc -l       # hide "Permission denied" noise
echo "deployed" | sudo tee -a /var/log/deploy.log   # append to a root-owned file
```

> **Warning:** `sudo echo hi > /etc/file` fails, because the redirection is done by *your* shell before `sudo` runs. Use `echo hi | sudo tee /etc/file`.

7.2 Practice data

Create a sample web-server log. Every example in this module uses it, so your output will match the manual.

```
mkdir -p ~/lab7 && cd ~/lab7
cat > access.log <<'EOF'
192.168.1.10 - - [16/Oct/2026:09:00:01 +0100] "GET /api/users HTTP/1.1" 200 512 "-" "curl/8.5.0"
0.012
192.168.1.11 - - [16/Oct/2026:09:00:03 +0100] "POST /api/login HTTP/1.1" 401 64 "-" "Mozilla/5.0"
0.030
192.168.1.10 - - [16/Oct/2026:09:00:05 +0100] "GET /api/orders HTTP/1.1" 200 2048 "-" "curl/8.5.0"
0.120
10.0.0.5 - - [16/Oct/2026:09:01:10 +0100] "GET /admin HTTP/1.1" 404 153 "-" "sqlmap/1.8" 0.002
192.168.1.12 - - [16/Oct/2026:09:01:12 +0100] "GET /api/users HTTP/1.1" 200 530 "-" "Mozilla/5.0"
0.015
192.168.1.11 - - [16/Oct/2026:09:01:20 +0100] "POST /api/login HTTP/1.1" 200 128 "-" "Mozilla/5.0"
0.041
10.0.0.5 - - [16/Oct/2026:09:01:21 +0100] "GET /.env HTTP/1.1" 404 153 "-" "sqlmap/1.8" 0.001
```

```
192.168.1.13 - - [16/Oct/2026:09:02:00 +0100] "GET /api/orders HTTP/1.1" 500 89 "-" "Mozilla/5.0"
1.502
192.168.1.10 - - [16/Oct/2026:09:02:30 +0100] "DELETE /api/orders/7 HTTP/1.1" 204 0 "-" "curl/8.5.0"
0.025
10.0.0.5 - - [16/Oct/2026:09:02:31 +0100] "GET /wp-login.php HTTP/1.1" 404 153 "-" "sqlmap/1.8"
0.001
192.168.1.12 - - [16/Oct/2026:09:03:00 +0100] "GET /api/products HTTP/1.1" 200 4096 "-"
"Mozilla/5.0" 0.210
192.168.1.13 - - [16/Oct/2026:09:03:05 +0100] "GET /api/orders HTTP/1.1" 502 157 "-" "Mozilla/5.0"
3.001
EOF
```

Each line is in Nginx “combined” format plus a final response time. With whitespace as the separator, the fields are:

Field	Content
`$1`	Client IP
`$4`	Time
`$6`	Method (with a leading `"`)
`$7`	Path
`$9`	Status
`$10`	Bytes sent
`$NF`	Response time (the last field)

7.3 grep — find lines

```
grep 'POST' access.log                  # lines containing POST
grep -i 'mozilla' access.log            # ignore case
grep -c '" 404 ' access.log             # count matching lines
grep -v 'curl' access.log               # lines NOT matching
grep -n '/api/orders' access.log        # with line numbers
grep -E '" (4|5)[0-9]{2} ' access.log  # extended regex: 4xx and 5xx
grep -o '/api/[a-z]*' access.log | sort -u    # print only the matching part
grep -rn 'DB_HOST' ~/practice           # search a directory tree
grep -A2 -B1 '" 500 ' access.log        # context: 1 line before, 2 after
```

Counting errors:

```
grep -cE '" (4|5)[0-9]{2} ' access.log

6
```

7.4 Regular expressions in one table

Pattern	Matches	Example
`.`	Any one character	`h.t` → hat, hot
`*`	Previous item 0 or more times	`ab*c` → ac, abbc
`+`, `?`	1 or more; 0 or 1 (ERE: `grep -E`)	`[0-9]+`
`^`, `$`	Start, end of line	`^#` comment lines
`[abc]`, `[^abc]`	One of; none of	`[aeiou]`
`[0-9]`, `[A-Za-z]`	Ranges	
`{n}`, `{n,m}`	Repetition (ERE)	`[0-9]{3}`
`(a|b)`	Alternation (ERE)	`(GET|POST)`
`\b`	Word boundary (GNU)	`\berror\b`

Basic regular expressions (plain `grep`, `sed`) need backslashes for `+ ? { } ( ) |`. Extended ones (`grep -E`, `sed -E`, `awk`) do not. Use `-E` and save yourself the backslashes.

7.5 cut, sort, uniq, wc, tr, paste

Top status codes:

```
awk '{print $9}' access.log | sort | uniq -c | sort -rn

      5 200
      3 404
      1 502
      1 500
      1 401
      1 204
```

Read the pipeline left to right:

1. `awk` takes column 9.
2. `sort` puts identical codes next to each other.
3. `uniq -c` counts each run of identical lines.
4. `sort -rn` puts the biggest counts first.

The order of tied counts can differ between systems.

```
cut -d'"' -f6 access.log | sort -u                # unique user agents (field 6 when split on ")
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -3   # top client IPs
wc -l access.log
echo "Hello World" | tr 'a-z' 'A-Z'               # translate characters
echo "a,b,,c" | tr -s ','                         # squeeze repeats
awk '{print $1}' access.log | sort -u | paste -sd,   # join lines with commas
```

7.6 sed — the stream editor

`sed` edits text as it flows past, and is ideal for scripted config changes. Create a sample config:

```
cat > app.env <<'EOF'
# Application settings
DEBUG=true
DB_HOST=localhost
DB_PORT=5432
API_URL=http://dev.api.example.com

SECRET_KEY=change-me
EOF

sed 's/^DEBUG=.*/DEBUG=false/' app.env           # substitute (prints the result, file unchanged)
sed -e '/^#/d' -e '/^$/d' app.env                # delete comment and blank lines
sed -n '2,4p' app.env                            # print only lines 2-4
sed -E 's#http://dev\.(api\.example\.com)#https://\1#' app.env   # capture group, custom delimiter
sed -i.bak 's/^DEBUG=.*/DEBUG=false/' app.env    # edit in place, keep app.env.bak

API_URL=https://api.example.com
```

> **Note:** always test a `sed` expression without `-i` first. With `-i.bak` you get a backup automatically.

7.7 awk — columns and calculations

`awk` processes each line as fields `$1` … `$NF` and runs `pattern { action }` rules. `BEGIN` runs before the first line and `END` after the last.

```
awk '$9 >= 500 {print $4, $7, $9}' access.log                 # server errors
awk '$NF > 1 {print $7, $NF"s"}' access.log                   # slow requests (> 1 s)
awk '{sum += $NF; n++} END {printf "requests=%d avg=%.3fs\n", n, sum/n}' access.log
awk -F= '/^DB_/ {print $1 " -> " $2}' app.env                 # custom separator
awk -F: '$3 >= 1000 {print $1}' /etc/passwd                   # human users

[16/Oct/2026:09:02:00 /api/orders 500
[16/Oct/2026:09:03:05 /api/orders 502
/api/orders 1.502s
/api/orders 3.001s
requests=12 avg=0.413s
```

Bytes served per endpoint, using an associative array:

```
awk '{bytes[$7] += $10} END {for (p in bytes) print bytes[p], p}' access.log | sort -rn | head -3
```

7.8 xargs — turning lines into arguments

```
find . -name "*.bak" | xargs rm -f                          # delete every listed file
find . -name "*.log" -print0 | xargs -0 gzip                # safe with spaces in names
printf '%s\n' web1 web2 | xargs -I{} echo "deploying to {}"
```

7.9 jq — JSON on the command line

APIs, Docker, `cast` and many cloud tools speak JSON. `jq` is the `awk` of JSON.

```
cat > users.json <<'EOF'
[{"id":1,"name":"Ada","role":"admin","active":true},
 {"id":2,"name":"Bayo","role":"dev","active":false},
 {"id":3,"name":"Chi","role":"dev","active":true}]
EOF
jq '.' users.json                                       # pretty-print
jq '.[] | select(.active) | .name' users.json           # filter
jq -r '.[] | "\(.id)\t\(.name)\t\(.role)"' users.json # raw TSV output
jq 'map(select(.role=="dev")) | length' users.json     # count
curl -s https://api.github.com/repos/torvalds/linux | jq '{stars: .stargazers_count,
lang: .language}'

"Ada"
"Chi"
```

Lab 7 — Log analyser

Using only pipelines over `access.log`, produce:

1. The number of requests per HTTP method.
2. Every IP that triggered a 404, with its count. (The answer identifies a scanner.)
3. The average response time of `/api/orders` only.
4. A one-line report: `total=12 errors=6 error_rate=50.0%`.
5. Save the whole report to `report.txt` **and** show it on screen at the same time.

Stretch: run the same pipeline on a real log, `/var/log/nginx/access.log`, after Module 16.

Checkpoint 7

1. Why must `2>&1` come *after* `> file`?
2. Why does `sudo echo x > /etc/f` fail, and what is the fix?
3. Why do you `sort` before `uniq -c`?
4. Write a `grep` that matches lines starting with `ERROR` or `WARN`.
5. When do you reach for `awk` instead of `cut`?

> **Tutor note:** 6 hours across two sessions. This module is the biggest productivity jump in the manual. Sources: *Bash Guide for Beginners*, chapters 4–6 (regular expressions, sed, awk); *Linux Command Line and Shell Scripting Bible*, chapters 19–22. Make learners build pipelines one stage at a time, checking each stage's output before adding the next.

Module 8 — Processes, jobs and signals

Objectives: list and inspect processes; read `top`/`htop`; send signals correctly; manage background jobs; set priorities; use `/proc` to see what the kernel sees.

8.1 What a process is

A process is a running program with:

- a **PID**;
- a parent (**PPID**);
- an owner;
- memory, open files and a state.

PID 1 is `systemd`, the ancestor of every process. When you run `python3 app.py` from Bash, Bash forks a child process and that child executes Python.

> **Mental model:** processes form a family tree. Killing a parent usually orphans its children, and systemd adopts the orphans. `pstree` draws the family.

8.2 Inspecting processes

```
ps aux                          # every process: user, PID, %CPU, %MEM, command
ps aux --sort=-%mem | head -6   # top 5 by memory
ps -ef --forest | less          # tree view with parent PIDs
pgrep -a python                 # PIDs and command lines matching "python"
pstree -p | head -20
top                             # live view
htop                            # friendlier live view (F6 sort, F9 kill, F4 filter)
```

Key columns in `top`/`htop`:

Column	Meaning
`load average: 0.52 0.40 0.31`	Runnable plus waiting tasks over the last 1, 5 and 15 minutes. Compare it with your CPU count (`nproc`).
`%CPU`	100% = one full core
`RES`	Physical memory actually used
`VIRT`	Virtual address space (much larger; rarely what you want)
`S` (state)	`R` running, `S` sleeping, `D` uninterruptible (usually disk I/O), `Z` zombie, `T` stopped

8.3 Signals and kill

Signal	Number	Meaning	Typical use
SIGTERM	15	Please exit cleanly (default)	Normal stop; the app can close connections first
SIGINT	2	Interrupt (`Ctrl+C`)	Stop a foreground program
SIGHUP	1	Hang up	Many daemons reload their config
SIGKILL	9	Kill immediately; cannot be caught	Last resort
SIGSTOP / SIGCONT	19 / 18	Pause / resume	Debugging

```
sleep 600 &                 # start a test process in the background
pgrep -a sleep
```

```
kill <PID>                  # SIGTERM
kill -HUP <PID>             # SIGHUP
pkill -f "python3 app.py"   # by command line
killall sleep               # by name
kill -9 <PID>               # only when SIGTERM fails
```

> **Warning:** `kill -9` gives the program no chance to flush data or release locks. A database killed with `-9` may need recovery on the next start. Try SIGTERM, wait, then escalate.

8.4 Foreground, background and jobs

```
sleep 300            # foreground; press Ctrl+Z to stop it
bg                   # resume it in the background
jobs                 # list jobs: [1]+ Running  sleep 300 &
fg %1                # bring it back to the foreground
nohup ./long.sh > long.log 2>&1 &   # survives logout
disown %1            # detach an existing job from the shell
```

For long tasks on remote servers, use a terminal multiplexer such as `tmux` so they survive dropped SSH connections:

```
sudo apt install tmux
tmux new -s work      # Ctrl+B then D to detach
tmux attach -t work
```

For anything permanent, write a systemd service (Module 9) instead of using `nohup`.

8.5 Priorities

`nice` values run from −20 (highest priority) to 19 (lowest). Only root can raise a priority.

```
nice -n 10 tar czf backup.tgz ~/projects      # start at low priority
renice -n 5 -p <PID>                          # change a running process
ionice -c3 rsync -a src/ dst/                 # disk I/O only when the disk is idle
```

8.6 /proc — the kernel's window

```
cat /proc/cpuinfo | grep "model name" | head -1
cat /proc/meminfo | head -5
cat /proc/loadavg
ls /proc/$$/              # $$ is the PID of your current shell
cat /proc/$$/status | head -10
sudo ls -l /proc/<PID>/fd # files a process has open
```

8.7 What is using my port, file or disk?

```
sudo ss -tulpn                  # listening sockets with process names
sudo lsof -i :8000              # process on port 8000
sudo lsof /var/log/syslog       # who has a file open
sudo fuser -v 8000/tcp
```

A classic backend problem is *"Error: address already in use"* when starting a dev server. Find the process with `lsof -i :8000`, check what it is, then stop it cleanly with SIGTERM.

Lab 8 — Process detective

1. Start three background jobs: `sleep 1000`, `yes > /dev/null`, and `python3 -m http.server 8000`.
2. Find which one is using the CPU, using `top` and then `ps --sort`.
3. Lower the priority of the CPU hog to 19 and watch `%CPU` in `htop`.
4. Find the PID listening on port 8000 with both `ss` and `lsof`.
5. Stop everything with the right signals, and explain why you chose each one.

Checkpoint 8

1. What does a load average of 4.0 mean on a 2-core machine? On an 8-core machine?
2. What is the difference between SIGTERM and SIGKILL?
3. What does process state `D` usually mean?
4. Why is `nohup` a poor way to run a production web app?

5. How do you find which program is listening on port 5432?

> **Tutor note:** 3 hours. Sources: *Introduction to Linux*, chapter 4; *Bash Guide for Beginners*, chapter 12 (signals); *Linux Command Line and Shell Scripting Bible*, chapters 4 and 16. Demonstrate a zombie by killing a parent that ignores SIGCHLD; it makes the state table memorable.

Module 9 — systemd: services, logs, timers and boot

Objectives: manage services with `systemctl`; read logs with `journalctl`; write your own service and timer units; understand the boot sequence well enough to troubleshoot it.

9.1 What systemd does

systemd is PID 1. It starts and supervises services, mounts filesystems, manages logging (the **journal**), scheduled jobs (**timers**) and the boot sequence (**targets**). Each thing it manages is a **unit**, described by a small text file.

Unit type	Example	Purpose
`.service`	`nginx.service`	A program to run and supervise
`.timer`	`apt-daily.timer`	A schedule that triggers a service
`.socket`	`ssh.socket`	Start a service when a connection arrives
`.target`	`multi-user.target`	A group of units; replaces SysV runlevels
`.mount`	`home.mount`	A mounted filesystem

Unit files live in:

- `/usr/lib/systemd/system/` — installed by packages; don't edit.
- `/etc/systemd/system/` — yours and administrators'; these take priority.

> **Note:** *The Linux System Administrator's Guide* (chapters 8–9) describes SysV `init`, `/etc/inittab` and runlevels. Modern Mint and Ubuntu replaced these with systemd: `runlevel 3` corresponds to `multi-user.target` and `runlevel 5` to `graphical.target`.

9.2 Managing services

```
systemctl status ssh                # state, PID, recent log lines
sudo systemctl start nginx
sudo systemctl stop nginx
sudo systemctl restart nginx        # full restart
sudo systemctl reload nginx         # re-read config without dropping connections
sudo systemctl enable --now nginx   # start now and at every boot
sudo systemctl disable nginx
systemctl is-active nginx; systemctl is-enabled nginx
systemctl list-units --type=service --state=running
systemctl --failed                  # what is broken right now
systemctl cat nginx                 # show the unit file actually in use
sudo systemctl edit nginx           # create an override (drop-in) safely
sudo systemctl mask cups            # make a service impossible to start
```

Reading `systemctl status`:

```
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-10-16 09:00:01 WAT; 2h ago
   Main PID: 1234 (nginx)
      Tasks: 3 (limit: 9387)
     Memory: 4.2M
```

- **Loaded** shows which file is used and whether the service is *enabled* at boot.
- **Active** shows whether it is running now.

These two are independent: a service can be enabled but stopped, or running but not enabled.

9.3 The journal

```
journalctl -u nginx                 # one service
journalctl -u nginx -f              # follow live
```

```
journalctl -u nginx --since "1 hour ago"
journalctl -u nginx --since "2026-10-16 09:00" --until "2026-10-16 10:00"
journalctl -b                          # since the current boot
journalctl -b -1 -p err                # errors from the previous boot
journalctl -p warning..alert           # priority range
journalctl -k                          # kernel messages (like dmesg)
journalctl -u myapi -o json-pretty | less
journalctl --disk-usage
sudo journalctl --vacuum-time=14d      # delete entries older than 14 days
```

Ubuntu and Mint also keep classic text logs through `rsyslog`:

File	Content
`/var/log/syslog`	General system messages
`/var/log/auth.log`	Logins, sudo, SSH
`/var/log/kern.log`	Kernel messages
`/var/log/apt/`	Package history
`/var/log/nginx/`	Web server logs (after Module 16)

logrotate compresses and removes old log files on a schedule (configuration in `/etc/logrotate.d/`).

9.4 Writing a service unit

Create a tiny service that writes a heartbeat line every 10 seconds.

1. The script:

```
sudo tee /usr/local/bin/heartbeat.sh > /dev/null <<'EOF'
#!/usr/bin/env bash
while true; do
  echo "heartbeat $(date --iso-8601=seconds) load=$(cut -d' ' -f1 /proc/loadavg)"
  sleep 10
done
EOF
sudo chmod 755 /usr/local/bin/heartbeat.sh
```

2. The unit file:

```
sudo tee /etc/systemd/system/heartbeat.service > /dev/null <<'EOF'
[Unit]
Description=Heartbeat demo service
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/heartbeat.sh
Restart=on-failure
RestartSec=5
User=nobody
NoNewPrivileges=true

[Install]
WantedBy=multi-user.target
EOF
```

3. Load, start and check it:

```
sudo systemctl daemon-reload
sudo systemctl enable --now heartbeat
journalctl -u heartbeat -f

Oct 16 10:00:00 mint-lab heartbeat.sh[4321]: heartbeat 2026-10-16T10:00:00+01:00 load=0.21
```

Key directives you will reuse for backend apps in Module 16:

Directive	Meaning
`After=network-online.target` plus `Wants=network-online.target`	Start after the network is actually up
`User=`, `Group=`	Never run apps as root

Directive	Meaning
`WorkingDirectory=`	Directory the program starts in
`EnvironmentFile=`	Load `KEY=value` pairs (secrets) from a root-readable file
`ExecStart=`	Absolute path to the program and its arguments
`Restart=on-failure` / `always`	Self-healing
`ProtectSystem=strict`, `ReadWritePaths=`, `PrivateTmp=true`	Sandboxing

`systemd-analyze security heartbeat` scores how well a service is sandboxed.

9.5 Timers: the modern cron

Run a cleanup every day at 02:30, catching up if the machine was off:

```
sudo tee /etc/systemd/system/cleanup.service > /dev/null <<'EOF'
[Unit]
Description=Delete temp files older than 7 days

[Service]
Type=oneshot
ExecStart=/usr/bin/find /var/tmp/myapp -type f -mtime +7 -delete
EOF

sudo tee /etc/systemd/system/cleanup.timer > /dev/null <<'EOF'
[Unit]
Description=Daily cleanup

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true

[Install]
WantedBy=timers.target
EOF

sudo mkdir -p /var/tmp/myapp
sudo systemctl daemon-reload
sudo systemctl enable --now cleanup.timer
systemctl list-timers
systemd-analyze calendar "Mon..Fri *-*-* 09:00"   # check what a schedule means
```

Cron is still available and is covered in Module 13. Timers add logging in the journal, catch-up after downtime (`Persistent=`) and dependencies.

9.6 The boot sequence

1. **Firmware** (UEFI/BIOS) runs its self-test and loads the boot loader from the EFI partition.
2. **GRUB 2** shows the menu and loads the kernel and the **initramfs**.
3. The **kernel** initialises hardware and mounts the initramfs, which finds and mounts the real root filesystem.
4. **systemd** (PID 1) starts units in parallel until it reaches the default target (`graphical.target` on Mint, `multi-user.target` on servers).

```
systemctl get-default
systemd-analyze                       # total boot time
systemd-analyze blame | head -10      # slowest units
systemd-analyze critical-chain
```

GRUB settings are in `/etc/default/grub`. After editing, run `sudo update-grub`. To boot into a rescue shell, edit the kernel line at the GRUB menu (`e`) and append `systemd.unit=rescue.target`.

> **Note:** *The Linux System Administrator's Guide* describes LILO. Every current Mint and Ubuntu release uses GRUB 2.

Lab 9 — Service owner

1. Create the `heartbeat` service from 9.4 and confirm it survives a reboot.
2. Make it crash: edit the script to run `exit 1` after the first heartbeat. Observe `Restart=on-failure` in `journalctl`, then restore it.
3. Replace `User=nobody` with a dedicated system user, and add `ProtectSystem=strict` and `PrivateTmp=true`. Compare the scores from `systemd-analyze security` before and after.
4. Create a timer that runs `df -h / >> /var/log/disk-report.log` every 15 minutes, and verify it with `systemctl list-timers`.
5. List the five slowest units at boot.

Checkpoint 9

1. What is the difference between `enable` and `start`?
2. Where do you put your own unit files, and why not in `/usr/lib/systemd/system`?
3. How do you see the errors from the previous boot?
4. Give two advantages of systemd timers over cron.
5. What does `daemon-reload` do, and when do you need it?

> **Tutor note:** 4 hours. This module replaces the SysV material in *The Linux System Administrator's Guide* (chapters 8–10) and *Introduction to Linux* (chapter 4) with the systemd tools that employers expect. Sources: the official systemd man pages `systemd.service(5)`, `systemd.timer(5)` and `journalctl(1)`.

Module 10 — Storage, filesystems, archives and backups

Objectives: inspect disks; partition, format and mount a disk; configure `/etc/fstab`; manage swap; understand LVM; find what is filling a disk; create archives; design and test backups.

10.1 Seeing your storage

```
lsblk -f                # block devices, filesystems, labels, mount points
df -hT                  # usage per mounted filesystem, with type
df -i                   # inode usage (a disk can be "full" with free space)
findmnt                 # mount tree
sudo blkid              # UUIDs and types
sudo fdisk -l           # partition tables
```

Device names:

Name	Device
`/dev/sda`, `/dev/sdb`	SATA/USB disks
`/dev/nvme0n1`	NVMe SSDs
`/dev/vda`	Virtual disks

Partitions add a number: `sda1`, `nvme0n1p1`.

10.2 Safe partitioning practice with a loop device

Practise on a file that pretends to be a disk, so a mistake costs nothing:

```
truncate -s 512M ~/lab.img
LOOP=$(sudo losetup --find --show --partscan ~/lab.img)
echo "$LOOP"                                    # e.g. /dev/loop0
sudo parted -s "$LOOP" mklabel gpt mkpart data ext4 1MiB 100%
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS "$LOOP"
sudo mkfs.ext4 -L labdata "${LOOP}p1"
sudo mkdir -p /mnt/lab
sudo mount "${LOOP}p1" /mnt/lab
df -h /mnt/lab

NAME       SIZE TYPE FSTYPE MOUNTPOINTS
loop0      512M loop
└─loop0p1  510M part
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop0p1    462M   24K  426M   1% /mnt/lab
```

Clean up:

```
sudo umount /mnt/lab
sudo losetup -d "$LOOP"
```

On a real second disk the steps are the same: replace `$LOOP` with, for example, `/dev/sdb`.

> **Warning:** `mkfs` and `parted` on the wrong device destroy its data instantly. Always confirm the device with `lsblk` first. When you are working on a real disk, unplug other external drives.

10.3 Mounting permanently with /etc/fstab

Use the UUID, not `/dev/sdb1`, because device names can change between boots:

```
sudo blkid /dev/sdb1
sudo mkdir -p /data
echo 'UUID=<uuid-from-blkid>  /data  ext4  defaults,noatime  0  2' | sudo tee -a /etc/fstab
sudo systemctl daemon-reload
sudo mount -a           # test: no output means success
findmnt /data
```

> **Warning:** an fstab error can drop the next boot to an emergency shell. Always run `sudo mount -a` after editing. For removable or optional disks, add `nofail` to the options.

10.4 Swap

Ubuntu uses a swap file (`/swap.img`); Mint creates `/swapfile`. To add or resize swap:

```
swapon --show
free -h
sudo fallocate -l 4G /swapfile2
sudo chmod 600 /swapfile2
sudo mkswap /swapfile2
sudo swapon /swapfile2
echo '/swapfile2 none swap sw 0 0' | sudo tee -a /etc/fstab
```

On servers with plenty of RAM, lowering `vm.swappiness` (Module 19) reduces swapping.

10.5 LVM in brief

Logical Volume Management puts a flexible layer between disks and filesystems:

- **physical volumes** (`pvcreate`) are grouped into
- **volume groups** (`vgcreate`), which are divided into
- **logical volumes** (`lvcreate`) that can grow while mounted.

Ubuntu Server's guided install uses LVM by default.

```
sudo pvs; sudo vgs; sudo lvs
sudo lvextend -r -L +10G /dev/ubuntu-vg/ubuntu-lv    # grow the LV and its filesystem in one step
```

10.6 What is filling my disk?

```
df -h
sudo du -xh --max-depth=1 / 2>/dev/null | sort -h | tail -10
sudo du -sh /var/log/* | sort -h | tail
journalctl --disk-usage
sudo apt clean                          # remove the downloaded package cache
docker system df                        # after Module 17
sudo apt install ncdu && sudo ncdu -x / # interactive explorer
```

A full root filesystem breaks databases, logins and package installs, so monitor it (Lab 13 builds an alert).

10.7 Archives and compression

Command	Purpose
`tar -czf site.tar.gz site/`	Create a gzip archive
`tar -cJf site.tar.xz site/`	xz: smaller, slower
`tar --zstd -cf site.tar.zst site/`	zstd: fast with a good ratio
`tar -tf site.tar.gz`	List contents
`tar -xzf site.tar.gz -C /tmp/restore`	Extract into a directory
`zip -r site.zip site/`, `unzip site.zip`	Windows-friendly format
`gzip file`, `gunzip file.gz`, `zcat file.gz`	Single files

Memory aid: **c**reate, e**x**tract, lis**t**; **z** gzip, **J** xz; **f** is followed by the archive name.

10.8 Backups that actually work

Follow the **3-2-1 rule**:

- **3** copies of the data;
- on **2** different media;
- with **1** copy off-site.

Tool	Best for
Timeshift	System snapshots (Mint)
`rsync`	Mirroring directories locally or over SSH
`restic` / `borg`	Encrypted, deduplicated, versioned backups

Tool	Best for
`pg_dump`, `mysqldump`	Databases (Module 15)

Mirroring with `rsync`:

```
rsync -aAXh --delete --info=progress2 ~/projects/ /media/$USER/BackupDisk/projects/
rsync -az --delete -e ssh ~/projects/ backup@server:/backups/laptop/projects/
```

Option	Meaning
`-a`	Archive: permissions, times, links, recursion
`-A -X`	ACLs and extended attributes
`--delete`	Make the destination an exact mirror

Run with `-n` (dry run) first.

Versioned, encrypted backups with `restic`:

```
sudo apt install restic
restic init --repo /media/$USER/BackupDisk/restic
restic -r /media/$USER/BackupDisk/restic backup ~/projects ~/Documents
restic -r /media/$USER/BackupDisk/restic snapshots
restic -r /media/$USER/BackupDisk/restic restore latest --target /tmp/restore-test
```

> **Warning:** a backup you have never restored is a hope, not a backup. Schedule a monthly restore test.

Lab 10 — Storage engineer

1. Using a 1 GB loop-device image, create two partitions (ext4 and xfs; `sudo apt install xfsprogs`), mount both under `/mnt/lab1` and `/mnt/lab2`, and add `nofail` entries to `/etc/fstab` using UUIDs. Reboot your VM and confirm both are mounted.
2. Fill one to 100% with `fallocate`, observe the error messages, then free the space.
3. Back up `~/practice` with `restic`, delete a file, and restore it.
4. Find the five largest directories under `/var`.

Checkpoint 10

1. Why should `/etc/fstab` use UUIDs?
2. What does `df -i` reveal that `df -h` does not?
3. Explain the 3-2-1 rule.
4. What does `rsync --delete` do, and why should you dry-run it first?
5. What three layers make up LVM?

> **Tutor note:** 4 hours. Sources: *The Linux System Administrator's Guide*, chapters 5, 6 and 12 (disks, memory, backups); *Introduction to Linux*, chapter 9 (backup techniques); *Linux Command Line and Shell Scripting Bible*, chapter 8 (filesystems). Their tools (`dump`, floppies, tapes) are dated; their principles (full vs. incremental backups, testing restores) are not.

Module 11 — Networking essentials

Objectives: understand IP addressing, ports and DNS; inspect and configure interfaces on Mint (NetworkManager) and Ubuntu Server (Netplan); troubleshoot connectivity layer by layer; use SSH professionally; transfer files; configure a firewall.

11.1 Just enough theory

Concept	Meaning
IP address	Where a host is. `192.168.1.20/24`: the first 24 bits are the network
Private ranges	`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16` (not routable on the Internet)
Gateway	The router that reaches other networks
DNS	Turns names into IP addresses
Port	Which program on the host: 22 SSH, 80 HTTP, 443 HTTPS, 5432 PostgreSQL, 6379 Redis, 8545 Ethereum JSON-RPC, 30303 Ethereum peer-to-peer
TCP / UDP	Reliable connection / fire-and-forget datagrams
Loopback	`127.0.0.1` (`localhost`): this machine only
`0.0.0.0`	When a server listens on it: every interface, so reachable from outside

> **Mental model:** the IP address is the street address of a building, the port is the apartment number, DNS is the phone directory, and the gateway is the post office that forwards mail to other towns.

11.2 Inspecting the network (modern tools)

```
ip -br addr              # brief: interface, state, addresses
ip route                 # routing table; "default via" is the gateway
ip -br link
ss -tulpn                # listening TCP/UDP sockets and their programs (sudo for all names)
ss -tn state established # current TCP connections
resolvectl status        # DNS servers in use (systemd-resolved)
hostname -I

lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             192.168.1.20/24 fe80::a00:27ff:fe4e:66a1/64
```

> **Note:** *The Linux Network Administrator's Guide* (2001) uses `ifconfig`, `route` and `netstat`. Those come from the old `net-tools` package. Use `ip` and `ss` from `iproute2` instead.

11.3 Troubleshooting layer by layer

Work from the bottom up and stop at the first failure:

```
ip -br link                          # 1. is the interface UP?
ip -br addr                          # 2. do I have an IP address?
ip route | grep default              # 3. do I have a gateway?
ping -c 3 <gateway-ip>               # 4. can I reach the router?
ping -c 3 1.1.1.1                    # 5. can I reach the Internet by IP?
dig +short example.com               # 6. does DNS work? (sudo apt install dnsutils)
curl -sI https://example.com         # 7. does the application protocol work?
tracepath example.com                # where along the path it breaks
```

- If step 5 works but step 6 fails, the problem is DNS. Check `resolvectl status` and `/etc/resolv.conf`.
- If everything works locally but users can't reach your server, check what the server listens on (`ss -tulpn`), then the firewall (`ufw`), then the cloud security group.

11.4 Configuring interfaces

Linux Mint and Ubuntu Desktop — NetworkManager (`nmcli`):

```
nmcli device status
nmcli connection show
nmcli device wifi list
nmcli device wifi connect "<SSID>" password "<password>"
sudo nmcli connection modify "Wired connection 1" \
  ipv4.method manual ipv4.addresses 192.168.1.50/24 \
  ipv4.gateway 192.168.1.1 ipv4.dns "1.1.1.1 8.8.8.8"
sudo nmcli connection up "Wired connection 1"
```

Ubuntu Server — Netplan (YAML in `/etc/netplan/`):

```
# /etc/netplan/50-static.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
      addresses: [192.168.1.50/24]
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [1.1.1.1, 8.8.8.8]

sudo chmod 600 /etc/netplan/50-static.yaml
sudo netplan try        # applies the change and rolls back after 120 s unless you confirm
sudo netplan apply
```

Local name overrides go in `/etc/hosts`, for example `127.0.0.1 api.local`.

11.5 HTTP from the terminal

```
curl https://api.github.com/zen
curl -i https://example.com                    # include response headers
curl -sS -o /dev/null -w '%{http_code} %{time_total}s\n' https://example.com
curl -X POST http://localhost:8000/api/login \
     -H 'Content-Type: application/json' \
     -d '{"username":"ada","password":"secret"}'
curl -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/me
wget -c https://example.com/big.iso            # resume an interrupted download
```

11.6 SSH like a professional

```
sudo apt install openssh-server          # on the machine you want to reach
ssh-keygen -t ed25519 -C "amakundi@laptop"   # key pair in ~/.ssh/; set a passphrase
ssh-copy-id user@192.168.1.50            # install your public key on the server
ssh user@192.168.1.50
```

Save hosts in `~/.ssh/config` (permissions 600):

```
Host lab
    HostName 192.168.1.50
    User amakundi
    IdentityFile ~/.ssh/id_ed25519

Host prod
    HostName 203.0.113.10
    User deploy
    Port 22
```

Then just type `ssh lab`.

Task	Command
Run one remote command	`ssh lab 'df -h /'`
Copy files	`scp file.txt lab:/tmp/` or `rsync -avz dir/ lab:~/dir/`
Interactive file transfer	`sftp lab`
Local port forward (reach a remote DB safely)	`ssh -L 5433:localhost:5432 prod`, then connect to

Task	Command
	`localhost:5433`
Remote port forward (show a local dev server)	`ssh -R 8080:localhost:8000 lab`
Jump through a bastion host	`ssh -J bastion prod`
Keep agent keys for the session	`eval "$(ssh-agent)"; ssh-add`

> **Warning:** the private key (`id_ed25519`) never leaves your machine and must be mode 600. Only the `.pub` file is shared.

11.7 The firewall: UFW

UFW is a friendly front-end to the kernel's nftables firewall.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH                    # do this BEFORE enabling on a remote server
sudo ufw allow 80,443/tcp
sudo ufw allow from 192.168.1.0/24 to any port 5432 proto tcp   # LAN-only PostgreSQL
sudo ufw limit OpenSSH                    # rate-limit brute force
sudo ufw enable
sudo ufw status numbered
sudo ufw delete <number>
```

> **Warning:** on a remote server, enabling UFW without allowing SSH first locks you out. Cloud firewalls (security groups) sit in front of UFW; both must allow the port.
>
> **Note:** *The Linux Network Administrator's Guide* (chapters 9–11) explains `ipchains` and early `iptables`. The concepts (chains, NAT, masquerading) still apply. Today the kernel uses **nftables**; `sudo nft list ruleset` shows the rules UFW generated.

Lab 11 — Two-machine network

Use two VMs on the same network (a NAT Network or bridged adapter):

1. Give the server VM a static IP with Netplan, using `netplan try`.
2. Set up key-based SSH from your Mint VM with a `~/.ssh/config` alias.
3. On the server, run `python3 -m http.server 8000`. Reach it from Mint with `curl`, then block it with UFW and show that the connection fails.
4. Install PostgreSQL on the server, listening only on `localhost`, and reach it from Mint through an SSH tunnel.
5. Break DNS on purpose (put a wrong nameserver in Netplan), diagnose it with the layer-by-layer method, and fix it.

Checkpoint 11

1. Your app works with `curl localhost:8000` on the server but not from your laptop. List the three things you check, in order.
2. What is the difference between listening on `127.0.0.1` and on `0.0.0.0`?
3. Why is `netplan try` safer than `netplan apply` over SSH?
4. Explain what `ssh -L 5433:localhost:5432 prod` does.
5. Which command replaced `netstat -tulpn`?

> **Tutor note:** 5 hours. Sources: *The Linux Network Administrator's Guide*, chapters 1–6 and 9–12 (TCP/IP concepts, name resolution, firewalls), modernised; *Introduction to Linux*, chapter 10; your *Linux Mastery Guide*, Phase 5. Skip the PPP, SLIP, UUCP, NIS and news chapters of the network guide; they are of historical interest only.

Module 12 — Bash scripting I: the language

Objectives: write, run and debug Bash scripts; use variables, quoting, parameter expansion and arithmetic; make decisions with tests and `case`; loop over lists, numbers, files and lines; accept arguments and return meaningful exit codes.

12.1 Your first script

A script is a text file of commands, with a **shebang** line that says which interpreter runs it.

File: `greet.sh`

```
#!/usr/bin/env bash
# greet.sh - greet a user and report the shell
name="${1:-World}"
echo "Hello, ${name}!"
echo "Script: $0, arguments: $#, PID: $$"
echo "Today is $(date +%A), you are in $PWD"

mkdir -p ~/bin && cd ~/bin
nano greet.sh            # paste the script, save
chmod +x greet.sh
./greet.sh
./greet.sh Ada

Hello, World!
Script: ./greet.sh, arguments: 0, PID: 742
Today is Friday, you are in /home/amakundi/bin
Hello, Ada!
```

Your PID and date will differ.

Ways to run a script:

Command	What happens
`./greet.sh`	Runs in a new process using the shebang (needs `x` permission)
`bash greet.sh`	Runs in a new Bash process (no `x` needed)
`source greet.sh` or `. greet.sh`	Runs **inside the current shell**; variables and `cd` persist. Used for `.bashrc` and `.env` files

Put `~/bin` on your `PATH` so your scripts run from anywhere. Mint and Ubuntu's default `~/.profile` does this automatically if the directory exists at login.

12.2 Variables and quoting

```
name="Ada Lovelace"        # no spaces around =
echo 'single: $name'       # single quotes: completely literal
echo "double: $name"       # double quotes: variables and $(...) expand
echo "command: $(date +%F)"

single: $name
double: Ada Lovelace
```

> **Rule of thumb:** always write `"$var"` in double quotes. Unquoted variables are split on spaces and glob-expanded, which is the number-one source of Bash bugs. For example, `rm $file` with `file="My Report.txt"` tries to delete two files called `My` and `Report.txt`.

Special variables:

Variable	Meaning
`$0`	Script name
`$1` … `$9`, `${10}`	Positional arguments

Variable	Meaning
`$#`	Number of arguments
`"$@"`	All arguments, each kept separate (use this)
`$?`	Exit status of the last command
`$$`	PID of this shell
`$!`	PID of the last background job

Environment variables are variables exported to child processes:

```
export APP_ENV=production
printenv APP_ENV
env | sort | head
```

12.3 Parameter expansion

```
path="/var/log/nginx/access.log"
echo "${path##*/}"        # access.log            (strip the longest prefix up to /)
echo "${path%/*}"         # /var/log/nginx        (strip the shortest suffix from /)
echo "${path%.log}.gz"    # /var/log/nginx/access.gz
echo "${#path}"           # 25                    (length)
greeting="hello world"
echo "${greeting^}" "${greeting^^}" "${greeting/world/Linux}"   # Hello world HELLO WORLD hello
Linux
echo "${missing:-default}"        # use a default if unset or empty
port=${1:-8000}                   # common pattern for optional arguments
db=${DATABASE_URL:?DATABASE_URL must be set}   # stop with an error if unset
```

12.4 Arithmetic

```
echo $(( 7 / 2 )) $(( 7 % 2 )) $(( 2 ** 10 ))     # 3 1 1024  (integers only)
count=0; (( count++ )); echo "$count"
echo "scale=2; 7/2" | bc                          # 3.50  (decimals with bc)
```

12.5 Exit codes and && / ||

Every command returns an exit status: `0` means success, anything else means failure.

```
ls /nonexistent 2>/dev/null; echo "exit status: $?"     # exit status: 2
mkdir -p build && cd build                               # cd only if mkdir succeeded
grep -q "^DEBUG=true" .env || echo "debug is off"
```

Scripts should `exit 0` on success and use non-zero codes for errors. By convention, `2` means wrong usage.

12.6 Tests and if

Use `[[ … ]]` in Bash; it is safer than the older `[ … ]`, which *Bash Guide for Beginners* (chapter 7) also explains.

Test	True when
`-f file`, `-d dir`, `-e path`	File / directory / anything exists
`-r`, `-w`, `-x`	Readable / writable / executable
`-s file`	Exists and is not empty
`-z "$s"`, `-n "$s"`	String empty / not empty
`"$a" == "$b"`, `!=`	String equality (== with a pattern: `[[ $f == *.log ]]`)
`$s =~ regex`	Regular-expression match
`-eq -ne -lt -le -gt -ge`	Integer comparison (or use `(( a > b ))`)
`&&`, `||`, `!`	AND, OR, NOT inside `[[ ]]`

```
if [[ -f /etc/nginx/nginx.conf ]]; then
  echo "nginx is installed"
elif command -v apache2 >/dev/null; then
  echo "apache is installed"
else
```

```
  echo "no web server found"
fi
```

12.7 case

```
file="report.pdf"
case "$file" in
  *.pdf)        echo "PDF document" ;;
  *.jpg|*.png)  echo "image" ;;
  *)            echo "other" ;;
esac
```

`case` is perfect for sub-commands in scripts: `start|stop|restart|status`.

12.8 Loops

File: `loops.sh`

```
#!/usr/bin/env bash
for env in dev staging prod; do
  echo "Deploying to $env"
done
for i in {1..3}; do
  echo "Attempt $i"
done
for (( i = 0; i < 3; i++ )); do
  printf 'i=%d\n' "$i"
done
count=0
while (( count < 3 )); do
  (( count++ ))
done
echo "count=$count"
while IFS=: read -r user _ uid _; do
  (( uid >= 1000 && uid < 60000 )) && echo "human user: $user ($uid)"
done < /etc/passwd
```

The last loop reads `/etc/passwd` one line at a time, splitting on `:`. `while IFS=… read -r` is the correct way to read lines; `for line in $(cat file)` splits on spaces and is wrong.

Loop control: `break` leaves the loop; `continue` skips to the next iteration.

12.9 Arrays

```
servers=(web1 web2 db1)
echo "${servers[0]}"        # web1
echo "${#servers[@]}"       # 3
servers+=(cache1)
for s in "${servers[@]}"; do ssh "$s" uptime; done

declare -A port=([ssh]=22 [http]=80 [postgres]=5432)    # associative array (dictionary)
for name in "${!port[@]}"; do echo "$name=${port[$name]}"; done
```

12.10 Input

```
read -rp "Project name: " project
read -rsp "Password: " pw; echo        # -s hides typing
read -rp "Continue? [y/N] " ans
[[ $ans =~ ^[Yy]$ ]] || exit 1
```

12.11 A complete small tool

This script validates its arguments, uses Bash's built-in `/dev/tcp` to test a port, and returns meaningful exit codes:

File: `check_port.sh`

```
#!/usr/bin/env bash
# check_port.sh HOST PORT - report whether a TCP port is open
if [[ $# -ne 2 ]]; then
  echo "Usage: $0 HOST PORT" >&2
  exit 2
fi
host=$1
```

```
port=$2
if [[ ! $port =~ ^[0-9]+$ ]] || (( port < 1 || port > 65535 )); then
  echo "Error: '$port' is not a valid port" >&2
  exit 2
fi
if timeout 3 bash -c "</dev/tcp/${host}/${port}" 2>/dev/null; then
  echo "OPEN    ${host}:${port}"
else
  echo "CLOSED  ${host}:${port}"
  exit 1
fi

./check_port.sh localhost 22
./check_port.sh localhost abc; echo "exit=$?"

OPEN    localhost:22
Error: 'abc' is not a valid port
exit=2
```

12.12 Debugging

```
bash -n script.sh          # syntax check only
bash -x script.sh          # trace: print each command before running it
set -x; …; set +x          # trace one part of a script
shellcheck script.sh       # static analysis (sudo apt install shellcheck)
```

ShellCheck catches quoting bugs, unused variables and portability problems. Every script in this manual passes ShellCheck with no warnings; make that your standard.

Lab 12 — Script toolkit

Write each of the following as a script in `~/bin`. Each must pass `shellcheck` and print a usage message when called wrongly.

1. `mkproject NAME`: creates `NAME/{src,tests,docs}` with `README.md` and `.gitignore`, and refuses if `NAME` already exists.
2. `extract FILE`: extracts `.tar.gz`, `.tar.xz`, `.zip` or `.gz` using `case`.
3. `users-report`: prints every human user (UID ≥ 1000) with their shell and home directory size.
4. `ports-check HOST PORT...`: checks several ports by reusing `check_port.sh`, and exits 1 if any is closed.

Checkpoint 12

1. What is the difference between `'$HOME'` and `"$HOME"`?
2. Why should you write `"$@"` rather than `$*`?
3. What do `${file%.txt}` and `${file##*/}` return for `file=/tmp/a/b.txt`?
4. What is the difference between running a script and `source`-ing it?
5. Why is `for line in $(cat file)` wrong, and what should you use instead?

> **Tutor note:** 6 hours. Sources: *Bash Guide for Beginners*, chapters 1–3 and 7–10; *Linux Command Line and Shell Scripting Bible*, chapters 11–15. Pair-program the first two lab scripts, then make learners present their own ShellCheck-clean scripts.

Module 13 — Bash scripting II: production-grade automation

Objectives: structure scripts with functions; parse options with `getopts`; fail safely with `set -Eeuo pipefail` and traps; prevent overlapping runs with locks; schedule work with cron and systemd timers; write the backup and monitoring scripts a junior admin is expected to own.

13.1 Functions

```
log() { printf '%s [%s] %s\n' "$(date '+%F %T')" "$1" "$2"; }

require_cmd() {
  local cmd=$1                          # local: does not leak out of the function
  command -v "$cmd" >/dev/null || { log ERROR "missing command: $cmd"; return 1; }
}

require_cmd jq && log INFO "jq found"
```

- Functions take arguments as `$1`, `$2` and so on, just like scripts.
- `return` sets an exit status (0–255).
- To "return" data, `echo` it and capture it with `$(…)`.

13.2 Strict mode and traps

```
#!/usr/bin/env bash
set -Eeuo pipefail
```

Option	Effect
`-e`	Exit when a command fails
`-u`	Treat unset variables as errors (catches typos)
`-o pipefail`	A pipeline fails if *any* stage fails, not just the last one
`-E`	ERR traps also fire inside functions

`set -e` ignores failures in `if` conditions and in commands followed by `||` or `&&`; that is how you handle expected failures.

Traps run code on exit, error or signal. They are ideal for cleaning up temporary files:

```
tmp=$(mktemp)
trap 'rm -f "$tmp"' EXIT
trap 'echo "failed at line $LINENO" >&2' ERR
trap 'echo "interrupted"; exit 130' INT TERM
```

13.3 Option parsing with getopts

This monitoring script is the model for every admin tool you write: usage text, options, validation, logging, a meaningful exit code, and no output in quiet mode unless there is a problem.

File: `disk_alert.sh`

```
#!/usr/bin/env bash
# disk_alert.sh - warn when any filesystem is above a usage threshold
set -Eeuo pipefail

THRESHOLD=80
QUIET=false

usage() {
  cat <<USAGE
Usage: ${0##*/} [-t PERCENT] [-q] [-h]
  -t PERCENT  alert threshold (default: ${THRESHOLD})
  -q          quiet: print only filesystems over the threshold
  -h          show this help
USAGE
}
```

```
log() { printf '%s [%s] %s\n' "$(date '+%F %T')" "$1" "$2"; }

while getopts ':t:qh' opt; do
  case $opt in
    t) THRESHOLD=$OPTARG ;;
    q) QUIET=true ;;
    h) usage; exit 0 ;;
    :) echo "Option -$OPTARG needs a value" >&2; usage >&2; exit 2 ;;
    \?) echo "Unknown option -$OPTARG" >&2; usage >&2; exit 2 ;;
  esac
done
shift $((OPTIND - 1))

[[ $THRESHOLD =~ ^[0-9]+$ ]] || { echo "Threshold must be a number" >&2; exit 2; }

alerts=0
while read -r fs pct mount; do
  use=${pct%\%}
  if (( use >= THRESHOLD )); then
    log WARN "$mount ($fs) is ${use}% full"
    alerts=$((alerts + 1))
  elif [[ $QUIET == false ]]; then
    log OK "$mount is ${use}% full"
  fi
done < <(df -P -x tmpfs -x devtmpfs -x squashfs -x overlay | awk 'NR > 1 {print $1, $5, $6}')

if (( alerts > 0 )); then
  exit 1
fi

./disk_alert.sh               # report all filesystems
./disk_alert.sh -t 90 -q      # only filesystems ≥ 90% full; exit 1 if any
./disk_alert.sh -x            # unknown option: prints usage, exit 2
```

```
2026-10-16 09:00:00 [OK] / is 58% full
2026-10-16 09:00:00 [WARN] /data (/dev/sdb1) is 92% full
```

`done < <(command)` is **process substitution**. It feeds the loop without a pipe, so variables changed inside the loop (`alerts`) keep their values afterwards. A pipe would run the loop in a subshell and lose them.

13.4 A production backup script

File: `backup.sh`

```
#!/usr/bin/env bash
# backup.sh SOURCE_DIR DEST_DIR [KEEP] - timestamped tar.gz backups with retention
set -Eeuo pipefail

SRC=${1:?usage: backup.sh SOURCE_DIR DEST_DIR [KEEP]}
DEST=${2:?usage: backup.sh SOURCE_DIR DEST_DIR [KEEP]}
KEEP=${3:-7}
STAMP=$(date +%Y%m%d_%H%M%S)
NAME="$(basename "$SRC")_${STAMP}.tar.gz"
TMP=$(mktemp)

log() { echo "$(date '+%F %T') $*"; }
cleanup() { rm -f "$TMP"; }
on_error() { log "ERROR: backup failed at line $1"; }
trap cleanup EXIT
trap 'on_error $LINENO' ERR

[[ -d $SRC ]] || { log "ERROR: $SRC is not a directory"; exit 1; }
mkdir -p "$DEST"

# one instance at a time
exec 9>"$DEST/.backup.lock"
flock -n 9 || { log "Another backup is running"; exit 1; }

log "Backing up $SRC"
tar -czf "$TMP" -C "$(dirname "$SRC")" "$(basename "$SRC")"
mv "$TMP" "$DEST/$NAME"
```

```
log "Created $DEST/$NAME ($(du -h "$DEST/$NAME" | cut -f1))"

# newest first; everything after the first KEEP entries is old
mapfile -t old < <(find "$DEST" -maxdepth 1 -name '*.tar.gz' -printf '%T@ %p\n' \
                   | sort -rn | tail -n +"$((KEEP + 1))" | cut -d' ' -f2-)
if (( ${#old[@]} > 0 )); then
  rm -f -- "${old[@]}"
  log "Removed ${#old[@]} old backup(s)"
fi
```

What makes this script production-grade:

- **Fails loudly.** `${1:?…}` aborts with a message if an argument is missing, and the ERR trap reports the line that failed.
- **Never leaves half-written backups.** It writes to a temporary file and renames it only when `tar` succeeds; the EXIT trap removes the temporary file on any failure.
- **Never runs twice at once.** `flock` takes an exclusive lock, so a slow run and the next scheduled run can't collide.
- **Keeps a fixed number of backups.** It lists backups newest first and deletes everything after the first `KEEP`.

```
./backup.sh ~/practice /var/backups/practice 7
tar -tzf /var/backups/practice/practice_*.tar.gz | head     # always verify a backup
```

13.5 Scheduling with cron

```
crontab -e          # your user's jobs
crontab -l
sudo crontab -e     # root's jobs
```

Cron has five time fields: minute (0–59), hour (0–23), day of month (1–31), month (1–12) and day of week (0–7, where 0 and 7 are Sunday).

```
# m  h  dom mon dow  command
*/5  *  *   *   *    /home/amakundi/bin/disk_alert.sh -q -t 85 >> /home/amakundi/disk.log 2>&1
30   2  *   *   *    /home/amakundi/bin/backup.sh /home/amakundi/projects /backup 14
0    9  *   *   1-5  /home/amakundi/bin/report.sh
@reboot              /home/amakundi/bin/on-boot.sh
```

Cron pitfalls:

- Cron runs with a **minimal environment**: use absolute paths and set `PATH=` at the top of the crontab.
- Output goes nowhere unless you redirect it.
- `%` must be escaped as `\%` inside crontab lines.
- System-wide jobs belong in `/etc/cron.d/` files, which need a user column.

For important jobs on servers, prefer a **systemd timer** (Module 9). It logs to the journal, catches up after downtime and is easy to monitor.

13.6 Notifications

Desktop (Mint):

```
notify-send "Backup finished" "practice backed up"
```

Server — send to a chat webhook (Slack, Discord or Telegram use similar JSON):

```
curl -fsS -X POST -H 'Content-Type: application/json' \
  -d "{\"text\":\"$(hostname): disk alert\"}" "$WEBHOOK_URL"
```

Combine the two: `./disk_alert.sh -q -t 85 || notify_webhook`.

13.7 Script style checklist

1. Shebang `#!/usr/bin/env bash` and `set -Eeuo pipefail`.
2. A comment header saying what the script does and how to call it.
3. A `usage` function, used for `-h` and for errors.
4. Every variable quoted; `local` inside functions.

5. Errors to stderr (`>&2`) with a non-zero exit code.
6. Temporary files from `mktemp`, removed by a trap.
7. Idempotent where possible: running it twice causes no harm.
8. Passes `shellcheck` with no warnings.
9. Kept in Git (Module 14).

Lab 13 — Automation engineer

1. Install `disk_alert.sh` and `backup.sh` in `~/bin`.
2. Schedule the backup of `~/projects` daily at 02:30 **with cron**, keeping 14 copies. Prove it works by temporarily scheduling it every minute.
3. Schedule `disk_alert.sh -q -t 85` every 10 minutes **with a systemd timer**, logging to the journal.
4. Write `service-health.sh SERVICE...`. It checks each service with `systemctl is-active`, restarts any that are down (only once), logs what it did, and exits 1 if a service is still down.
5. Write `rotate-logs.sh DIR DAYS`. It compresses `*.log` files older than DAYS with `gzip` and deletes `*.gz` files older than 30 days.

Checkpoint 13

1. What does each part of `set -Eeuo pipefail` do?
2. Why use `mktemp` and a trap rather than a fixed `/tmp/backup.tmp`?
3. What problem does `flock` solve in scheduled jobs?
4. Why does a variable changed inside `cmd | while read …` lose its value, and how does `done < <(cmd)` fix it?
5. List three differences between cron and systemd timers.

> **Tutor note:** 6 hours. Sources: *Bash Guide for Beginners*, chapters 10–12 (variables, functions, signals); *Linux Command Line and Shell Scripting Bible*, chapters 14–17 and 24–26; your *Linux Mastery Guide*, Phase 6. Your Mastery Guide's backup example uses `ls -t | tail | xargs rm`, which breaks on unusual file names and has no locking; compare it with the version above in class as a code-review exercise.

Level 3 — Backend engineer

Level 3 applies everything so far to the job you are training for: building and running backend services. You will set up a professional development workstation on Mint, run databases, deploy a Django (or Node) API to an Ubuntu server behind Nginx with HTTPS, containerise it, secure and monitor it, and automate deployment. Budget about eight weeks.

Module 14 — The backend developer's workstation

Objectives: configure Git and GitHub over SSH; manage Python versions and virtual environments the modern way; manage Node.js with `nvm`; handle environment variables and secrets; set up VS Code for Linux development.

14.1 Git and GitHub

```
sudo apt install git
git config --global user.name "Samuel Amakundi Bako"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global pull.rebase true
git config --global core.editor "nano"
git config --list --global
```

Connect to GitHub with an SSH key:

1. Create a key (Module 11) if you don't have one: `ssh-keygen -t ed25519 -C "github-laptop"`.
2. Show the public key with `cat ~/.ssh/id_ed25519.pub`, then add it in GitHub under **Settings → SSH and GPG keys**.
3. Test the connection:

```
ssh -T git@github.com

Hi <your-username>! You've successfully authenticated, but GitHub does not provide shell access.
```

Daily workflow:

```
git clone git@github.com:<user>/<repo>.git
git switch -c feature/login          # new branch
git status; git diff
git add -p                           # stage hunk by hunk
git commit -m "Add login endpoint"
git push -u origin feature/login
git pull                              # with rebase configured above
git log --oneline --graph --all | head -20
```

Install the GitHub CLI for pull requests from the terminal: `sudo apt install gh` (or use GitHub's apt repository for the latest version), then `gh auth login`.

> **Warning:** never commit secrets. Add `.env`, `*.pem` and `*.key` to `.gitignore` *before* the first commit. If a secret was pushed, **rotate it**; deleting it from history does not un-leak it. Scan repositories with `gitleaks` before pushing.

14.2 Python the right way on Ubuntu and Mint

The system `python3` belongs to the operating system. Since Ubuntu 23.04 (and therefore Mint 22), `pip install` into the system Python is blocked with an *externally-managed-environment* error (PEP 668). That is intentional: it protects tools such as Update Manager from being broken by your packages.

Need	Tool
Project dependencies	A **virtual environment** per project (`python3 -m venv` or `uv`)
Command-line tools (`black`, `httpie`, `pre-commit`)	`pipx install <tool>` or `uv tool install <tool>`
A Python version the OS doesn't ship	`uv python install 3.12`, or `pyenv`
System libraries for building packages	`apt` (`libpq-dev`, `python3-dev`, `build-essential`)

Standard venv workflow:

```
sudo apt install python3-venv python3-dev libpq-dev build-essential
mkdir -p ~/projects/myapi && cd ~/projects/myapi
python3 -m venv .venv
source .venv/bin/activate          # the prompt shows (.venv)
```

```
python -m pip install --upgrade pip
pip install django djangorestframework gunicorn "psycopg[binary]" dj-database-url
pip freeze > requirements.txt
deactivate
```

Faster alternative — uv (a Python package and project manager from Astral):

```
curl -LsSf https://astral.sh/uv/install.sh | sh
uv init myservice && cd myservice
uv add fastapi "uvicorn[standard]"
uv run uvicorn main:app --reload
```

> **Mental model:** a virtual environment is a private toolbox for one project. The system toolbox stays untouched, and two projects can use different versions of the same library without conflict.

14.3 Node.js with nvm

The `nodejs` package in Ubuntu's archive often lags behind. Use **nvm** for per-user, per-project Node versions:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.7/install.sh | bash
source ~/.bashrc
nvm install --lts          # the current LTS release
nvm use --lts
node -v; npm -v
echo "lts/*" > .nvmrc      # per-project version; then just `nvm use`
```

> **Note:** check github.com/nvm-sh/nvm for the latest installer version. Node.js 24 is the Active LTS line until late October 2026, when Node.js 26 becomes LTS. Production should run only even-numbered LTS releases. Hardhat 3 (Module 21) needs Node.js 22.13 or later.

For servers, the NodeSource or official Node.js binary packages are an alternative when an app must run as a system user under systemd.

14.4 Environment variables and secrets

Follow the twelve-factor principle: configuration lives in the environment, not in the code.

```
# .env  (git-ignored, mode 600)
DJANGO_SECRET_KEY=change-me
DJANGO_DEBUG=true
DATABASE_URL=postgres://myapi:secret@localhost:5432/myapi
```

Ways to load it:

- In Python: `python-dotenv` or `django-environ`.
- In the shell: `set -a; source .env; set +a`.
- In production: systemd `EnvironmentFile=` (Module 16).
- Automatically per directory: `direnv` (`sudo apt install direnv`, then add `eval "$(direnv hook bash)"` to `~/.bashrc`).

Commit a `.env.example` containing every key with dummy values, so teammates know what to set.

14.5 Editor and terminal setup

- **VS Code:** download the `.deb` from code.visualstudio.com and install with `sudo apt install ./code_*.deb`; it adds Microsoft's repository for updates. Useful extensions: Python, Pylance, Ruff, Remote - SSH, Docker, GitLens, ShellCheck, Solidity.
- **Terminal:** `tmux` for persistent sessions; optionally `zsh` with `oh-my-zsh`. Learn Bash first, because servers run Bash.
- **Database GUIs:** DBeaver (Flatpak) or pgAdmin.

Lab 14 — Workstation as code

1. Write `~/bin/bootstrap-dev.sh`. It must be idempotent (safe to run twice) and must:
   - install the APT packages from this module;
   - install `nvm` and the Node LTS;

   - install `uv` and `pipx`;
   - configure Git.
2. Create a Django project in a venv, with `requirements.txt`, `.env.example` and `.gitignore`, and push it to a new private GitHub repository over SSH.
3. Prove that `pip install requests` outside a venv is blocked, and explain why in one paragraph.

Checkpoint 14

1. Why does `pip install` outside a virtual environment fail on Ubuntu 24.04 and Mint 22?
2. What is the difference between `pipx install` and `pip install` inside a venv?
3. What is `.nvmrc` for?
4. What do you do if you accidentally push an API key to GitHub?

> **Tutor note:** 4 hours. This module extends Phase 7 of your *Linux Mastery Guide* with current tools: `uv`, PEP 668, and nvm v0.40. Make each learner run the bootstrap script on a fresh VM snapshot. A script that only works on the author's machine is a lesson in itself.

Module 15 — Databases and caches on Linux

Objectives: install, secure, back up and restore PostgreSQL; know the MariaDB/MySQL equivalents; run Redis safely; connect applications using least-privilege accounts.

15.1 PostgreSQL

```
sudo apt install postgresql postgresql-contrib
sudo systemctl enable --now postgresql
sudo -u postgres psql -c "SELECT version();"
```

PostgreSQL has its own users (**roles**). The `postgres` role is the superuser, and on Ubuntu the local Linux user `postgres` logs in to it with **peer authentication** (the Linux username must match the role).

Create an application role and database:

```
sudo -u postgres psql <<'SQL'
CREATE ROLE myapi WITH LOGIN PASSWORD 'use-a-long-random-password';
CREATE DATABASE myapi OWNER myapi;
SQL
psql "postgresql://myapi:use-a-long-random-password@localhost:5432/myapi" -c '\conninfo'
```

Key files (the version number in the path varies by release):

File	Purpose
`/etc/postgresql/<version>/main/postgresql.conf`	`listen_addresses`, memory settings
`/etc/postgresql/<version>/main/pg_hba.conf`	Who may connect, from where, and how (`peer`, `scram-sha-256`)
`/var/lib/postgresql/<version>/main/`	Data files
`journalctl -u postgresql`, `/var/log/postgresql/`	Logs

Find your version with `ls /etc/postgresql/`.

Security defaults to keep:

- `listen_addresses = 'localhost'` unless another server truly needs access.
- If it does, allow that one IP in `pg_hba.conf` with `scram-sha-256`, *and* in UFW.
- The app role owns only its own database and is never a superuser.
- Reach production databases from your laptop through an SSH tunnel, not an open port.

Useful `psql` commands:

Command	Purpose
`\l`	List databases
`\c db`	Connect to a database
`\dt`	List tables
`\d table`	Describe a table
`\du`	List roles
`\x`	Toggle expanded output
`\timing`	Show query times
`\q`	Quit

Backup and restore:

```
pg_dump -Fc -U myapi -h localhost myapi > myapi_$(date +%F).dump        # custom format, compressed
pg_restore -l myapi_2026-10-16.dump | head                              # inspect the contents
sudo -u postgres createdb myapi_restore
sudo -u postgres pg_restore -d myapi_restore --no-owner myapi_2026-10-16.dump
sudo -u postgres pg_dumpall --globals-only > roles.sql                  # roles and permissions
```

Store the password for scripts in `~/.pgpass` (mode 600), one line per server:
`localhost:5432:myapi:myapi:<password>`.

15.2 MariaDB / MySQL in brief

```
sudo apt install mariadb-server
sudo mariadb-secure-installation
sudo mariadb -e "CREATE DATABASE shop; CREATE USER 'shop'@'localhost' IDENTIFIED BY '<pw>'; GRANT
ALL ON shop.* TO 'shop'@'localhost';"
mariadb-dump -u shop -p shop > shop.sql
```

The configuration lives in `/etc/mysql/mariadb.conf.d/`. The concepts are the same as PostgreSQL: bind to localhost, one user per application, scheduled dumps.

15.3 Redis

```
sudo apt install redis-server
sudo systemctl enable --now redis-server
redis-cli ping                      # PONG
redis-cli set greeting hello EX 60  # key with a 60-second expiry
redis-cli get greeting
```

Hardening in `/etc/redis/redis.conf`:

- keep `bind 127.0.0.1 -::1`;
- set `requirepass` (or ACL users) if anything other than localhost can connect;
- set `maxmemory 256mb` and `maxmemory-policy allkeys-lru` for a cache.

> **Warning:** Redis exposed to the Internet without authentication is one of the most common server compromises. Never open port 6379 publicly.

15.4 Database chores to automate

- A nightly `pg_dump` with retention: adapt `backup.sh` from Module 13, run it from a systemd timer, and copy the dumps off the server.
- A monthly **restore test** into a scratch database.
- Watch disk space under `/var/lib/postgresql` (Module 19).
- Apply minor-version updates with `apt`. Major-version upgrades need `pg_upgradecluster` and a plan.

Lab 15 — Data tier

1. Install PostgreSQL and Redis on your Ubuntu Server VM.
2. Create the role and database `myapi` with a strong random password: `openssl rand -base64 24`.
3. Point your Django project's `DATABASES` setting at it via `DATABASE_URL`, run the migrations, and create a superuser.
4. Write `pg-backup.sh` and a systemd timer that dumps `myapi` nightly and keeps 14 dumps.
5. Drop a table on purpose, then restore it from the latest dump into a new database, and verify the row counts.
6. From your Mint machine, connect to the database through an SSH tunnel with DBeaver.

Checkpoint 15

1. What is peer authentication, and why can `sudo -u postgres psql` log in without a password?
2. Which two files control *where* PostgreSQL listens and *who* may connect?
3. Why use `pg_dump -Fc` instead of plain SQL for large databases?
4. What three settings make a Redis cache safe?

> **Tutor note:** 4 hours. Sources: the official PostgreSQL documentation (client authentication, backup and restore); Ubuntu Server documentation (databases); your *Linux Mastery Guide*, Phase 8.

Module 16 — Deploying a backend: Gunicorn, systemd, Nginx and HTTPS

Objectives: prepare an Ubuntu server; deploy a Django API under a dedicated user as a systemd service; put Nginx in front as a reverse proxy serving static files; enable HTTPS with Let's Encrypt; rotate logs; deploy updates with a script.

This module is the heart of the manual. When you can do it from memory, you can deploy most Python or Node backends.

16.1 The target architecture

```
Internet ──443/80──► Nginx (TLS, static files, buffering)
                        │  unix socket /run/myapi/gunicorn.sock
                        ▼
                  Gunicorn (3 workers) ── runs as user "myapi", managed by systemd
                        │
                        ▼
                  Django app ──► PostgreSQL (localhost:5432), Redis (localhost:6379)
```

> **Mental model:** Nginx is the receptionist. It greets every visitor, handles the secure envelope (TLS), hands out brochures (static files) itself, and passes only real requests to the back office (Gunicorn). Gunicorn is the office manager who keeps a fixed team of workers (your Django code) busy. systemd is the building manager who restarts the office if it ever goes dark.

16.2 Prepare the server

On a fresh Ubuntu Server (cloud or VM), as a sudo user:

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y python3-venv python3-dev build-essential libpq-dev nginx git
sudo ufw allow OpenSSH && sudo ufw allow 'Nginx Full' && sudo ufw enable
sudo timedatectl set-timezone Africa/Lagos
```

Create the application user and directories:

```
sudo adduser --system --group --home /srv/myapi --shell /usr/sbin/nologin myapi
sudo mkdir -p /srv/myapi/app /srv/myapi/media /etc/myapi
sudo chown -R myapi:myapi /srv/myapi
```

16.3 Get the code and dependencies

```
sudo -u myapi git clone https://github.com/<user>/myapi.git /srv/myapi/app
cd /srv/myapi/app
sudo -u myapi python3 -m venv .venv
sudo -u myapi .venv/bin/pip install -r requirements.txt
```

For a private repository, add a read-only **deploy key** (an SSH key for the `myapi` user, added to that one repository on GitHub).

16.4 Production settings

The Django settings must read everything sensitive from the environment. The relevant lines of `config/settings.py`:

```
import os
from pathlib import Path

SECRET_KEY = os.environ['DJANGO_SECRET_KEY']
DEBUG = os.environ.get('DJANGO_DEBUG', 'false').lower() == 'true'
ALLOWED_HOSTS = os.environ.get('DJANGO_ALLOWED_HOSTS', 'localhost').split(',')

STATIC_URL = "static/"
STATIC_ROOT = BASE_DIR / "staticfiles"
CSRF_TRUSTED_ORIGINS = os.environ.get("DJANGO_CSRF_TRUSTED_ORIGINS", "http://localhost").split(",")
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")
```

```
import dj_database_url      # pip install dj-database-url
DATABASES = {"default": dj_database_url.config(conn_max_age=600)}   # reads DATABASE_URL
```

Add a health-check endpoint in `config/urls.py`. Monitoring and deploy scripts call it:

```
from django.contrib import admin
from django.http import JsonResponse
from django.urls import path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("health/", lambda request: JsonResponse({"status": "ok"})),
]
```

Create the environment file, readable only by root and the app's group:

```
sudo tee /etc/myapi/myapi.env > /dev/null <<EOF
DJANGO_SECRET_KEY=$(openssl rand -base64 48)
DJANGO_DEBUG=false
DJANGO_ALLOWED_HOSTS=api.example.com
DJANGO_CSRF_TRUSTED_ORIGINS=https://api.example.com
DATABASE_URL=postgres://myapi:<db-password>@localhost:5432/myapi
EOF
sudo chown root:myapi /etc/myapi/myapi.env
sudo chmod 640 /etc/myapi/myapi.env
```

Run the one-off management commands with the same environment:

```
cd /srv/myapi/app
sudo -u myapi bash -c 'set -a; source /etc/myapi/myapi.env; set +a;
  .venv/bin/python manage.py migrate &&
  .venv/bin/python manage.py collectstatic --noinput &&
  .venv/bin/python manage.py check --deploy'
```

`check --deploy` lists the security settings still missing. Work through its warnings; most become irrelevant once HTTPS is on (16.7).

16.5 Gunicorn as a systemd service

```
# /etc/systemd/system/myapi.service
[Unit]
Description=myapi Django application (Gunicorn)
After=network-online.target postgresql.service
Wants=network-online.target

[Service]
Type=notify
User=myapi
Group=myapi
WorkingDirectory=/srv/myapi/app
EnvironmentFile=/etc/myapi/myapi.env
RuntimeDirectory=myapi
ExecStart=/srv/myapi/app/.venv/bin/gunicorn config.wsgi:application \
    --workers 3 \
    --bind unix:/run/myapi/gunicorn.sock \
    --access-logfile - \
    --error-logfile -
ExecReload=/bin/kill -s HUP $MAINPID
Restart=on-failure
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/srv/myapi/media

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now myapi
systemctl status myapi
journalctl -u myapi -f
```

Directive	Why it is there
`Type=notify`	Gunicorn tells systemd when it is ready, so dependent units start at the right time
`RuntimeDirectory=myapi`	systemd creates `/run/myapi` (owned by `myapi`) at every start; `/run` is wiped on reboot
`--workers 3`	A common starting point is (2 × CPU cores) + 1; measure and adjust
`ExecReload`	`systemctl reload myapi` gracefully restarts workers (zero-downtime code reload)
`ProtectSystem=strict`	The whole filesystem is read-only to the app except `ReadWritePaths`

Nginx (user `www-data`) must be able to reach the socket: `sudo usermod -aG myapi www-data`, then `sudo systemctl restart nginx`.

Node.js variant. The structure is the same; only the program changes:

```
ExecStart=/usr/bin/node /srv/myapp/app/dist/server.js
Environment=NODE_ENV=production PORT=3000
```

Nginx then proxies to `http://127.0.0.1:3000` instead of the unix socket.

16.6 Nginx reverse proxy

```
# /etc/nginx/sites-available/myapi
upstream myapi {
    server unix:/run/myapi/gunicorn.sock;
}

server {
    listen 80;
    server_name api.example.com;

    client_max_body_size 10M;

    location /static/ {
        alias /srv/myapi/app/staticfiles/;
        expires 7d;
    }

    location / {
        proxy_pass http://myapi;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/myapi /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t                 # ALWAYS test before reloading
sudo systemctl reload nginx
curl -s http://api.example.com/health/
```

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
{"status": "ok"}
```

This exact Django, Gunicorn and Nginx combination was tested end to end while preparing this manual.

Troubleshooting map:

Symptom	Likely cause	Check
`502 Bad Gateway`	Gunicorn not running, or Nginx can't	`systemctl status myapi`; `ls -l`

Symptom	Likely cause	Check
	reach the socket	`/run/myapi/`; is `www-data` in group `myapi`?
`504 Gateway Timeout`	The app is too slow	`proxy_read_timeout`, slow queries, worker count
`400 Bad Request` (Django)	Host not in `ALLOWED_HOSTS`	The environment file
CSS missing in the admin	`collectstatic` not run, or a wrong `alias` path (it needs the trailing slash)	`ls /srv/myapi/app/staticfiles`
`403` on static files	Nginx can't traverse the directories	`namei -l /srv/myapi/app/staticfiles/admin`

16.7 HTTPS with Let's Encrypt

This needs a domain whose DNS **A record** points to the server, with ports 80 and 443 open.

```
sudo snap install --classic certbot           # the method recommended by Certbot on Ubuntu
sudo ln -s /snap/bin/certbot /usr/bin/certbot
sudo certbot --nginx -d api.example.com
sudo certbot renew --dry-run                  # renewal runs automatically from a timer
```

Certbot edits the Nginx site to listen on 443 with the certificate and to redirect HTTP to HTTPS. Afterwards, add these lines to the Django environment file and restart the app:

```
DJANGO_SECURE_SSL_REDIRECT=true
DJANGO_SESSION_COOKIE_SECURE=true
DJANGO_CSRF_COOKIE_SECURE=true
```

Wire them into `settings.py` the same way as `DEBUG`.

> **Note:** `apt install certbot python3-certbot-nginx` also works and is the usual choice where Snap is unavailable, such as Linux Mint.

16.8 Logs and rotation

- **Application logs:** go to the journal, because Gunicorn writes to stdout and stderr: `journalctl -u myapi`.
- **Nginx logs:** `/var/log/nginx/access.log` and `error.log`, rotated by `/etc/logrotate.d/nginx`.
- **Files your app writes itself:** add a logrotate rule:

```
# /etc/logrotate.d/myapi
/srv/myapi/logs/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
}
```

Test a rule without changing anything: `sudo logrotate -d /etc/logrotate.d/myapi`.

16.9 A deploy script

```
#!/usr/bin/env bash
# /usr/local/bin/deploy-myapi - pull, install, migrate, reload, verify
set -Eeuo pipefail
APP=/srv/myapi/app
run() { sudo -u myapi bash -c "set -a; source /etc/myapi/myapi.env; set +a; cd $APP; $*"; }

echo "==> Pulling code";        sudo -u myapi git -C "$APP" pull --ff-only
echo "==> Dependencies";        run ".venv/bin/pip install -q -r requirements.txt"
echo "==> Migrations";          run ".venv/bin/python manage.py migrate --noinput"
echo "==> Static files";        run ".venv/bin/python manage.py collectstatic --noinput"
echo "==> Reloading";           sudo systemctl reload myapi
sleep 3
```

```
echo "==> Health check"
curl -fsS --max-time 5 https://api.example.com/health/ && echo " OK"
```

When the script fails, it stops before reloading the app. Module 20 runs it from GitHub Actions.

Lab 16 — Ship it

1. Deploy your Lab 15 Django project to your Ubuntu server, following 16.2–16.6 exactly.
2. Break it three ways and fix it from the logs alone:
   - stop the service;
   - remove `www-data` from the `myapi` group;
   - set a wrong `ALLOWED_HOSTS`.
3. If you have a domain, enable HTTPS and get a grade A at ssllabs.com/ssltest. If not, use a self-signed certificate (`openssl req -x509 …`) and explain the browser warning.
4. Run `deploy-myapi` after pushing a visible change, and confirm zero downtime by running `while true; do curl -s -o /dev/null -w '%{http_code}\n' https://…/health/; sleep 0.2; done` during the deploy.

Checkpoint 16

1. Why put Nginx in front of Gunicorn instead of exposing Gunicorn directly?
2. Why does the app run as `myapi` with `/usr/sbin/nologin` as its shell?
3. What causes a 502 error, and which three things do you check?
4. Why use `EnvironmentFile=` instead of writing secrets in the unit file or in Git?
5. What does `systemctl reload myapi` do differently from `restart`?

> **Tutor note:** 8 hours over two or three sessions. Sources: Gunicorn deployment documentation (systemd and Nginx examples), Nginx reverse-proxy documentation, Django deployment checklist, Certbot instructions, your *Linux Mastery Guide*, Phase 9. Learners should deploy at least twice from scratch: the second time without notes.

Module 17 — Containers with Docker

Objectives: install Docker Engine on Ubuntu and Mint; run, inspect and clean up containers; write efficient Dockerfiles; run a multi-service stack with Docker Compose; understand volumes, networks and the Docker/UFW firewall trap.

17.1 Why containers

A container packages an app with its userland (libraries and runtime) and runs it isolated on the host's kernel, using Linux namespaces and cgroups. It is lighter than a VM and reproducible across laptops, CI and servers.

> **Mental model:** an image is a recipe plus the pre-cooked ingredients, frozen. A container is a meal made from it: you can make many identical meals, and throwing one away doesn't affect the recipe. Volumes are the fridge, where data survives after the meal is gone.

17.2 Install Docker Engine (Ubuntu and Mint)

Use the repository set up in Module 6 (the `UBUNTU_CODENAME` version works on Mint), then:

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo docker run --rm hello-world
sudo usermod -aG docker "$USER"     # run docker without sudo (log out and in)
```

> **Warning:** membership of the `docker` group is equivalent to root on that machine, because anyone in it can mount `/` into a container. Give it only to trusted users. On shared servers consider rootless Docker or Podman.
>
> **Note:** Docker officially supports Ubuntu, not its derivatives; on Mint the Ubuntu packages work through the `UBUNTU_CODENAME` repository line. Do not install the old `docker.io` or `docker-compose` (v1) packages alongside these.

17.3 Everyday commands

```
docker run -d --name web -p 8080:80 nginx:stable     # run detached, publish a port
docker ps; docker ps -a                              # running / all containers
docker logs -f web
docker exec -it web bash                             # a shell inside the container
docker stop web && docker rm web
docker images; docker pull python:3.12-slim
docker system df; docker system prune               # disk usage and clean-up
docker inspect web | jq '.[0].NetworkSettings.IPAddress'
```

17.4 A production-style Dockerfile for the Django API

```
# Dockerfile
FROM python:3.12-slim AS base
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends libpq5 \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
RUN useradd --system --uid 10001 app && chown -R app /app
USER app

EXPOSE 8000
CMD ["gunicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000", "--workers", "3"]
```

```
# .dockerignore
.venv
.git
__pycache__
*.pyc
.env
staticfiles
```

Good practices shown here:

- a slim base image;
- dependencies copied before the code, so the slow `pip install` layer is cached;
- no secrets in the image;
- a non-root user;
- a single process per container.

```
docker build -t myapi:1.0 .
docker run --rm -p 8000:8000 --env-file .env myapi:1.0
```

17.5 Docker Compose: app + PostgreSQL + Redis

```
# compose.yaml
services:
  api:
    build: .
    env_file: .env
    ports:
      - "127.0.0.1:8000:8000"     # host-local only; Nginx on the host proxies to it
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_started
    restart: unless-stopped

  db:
    image: postgres:17
    environment:
      POSTGRES_DB: myapi
      POSTGRES_USER: myapi
      POSTGRES_PASSWORD: ${DB_PASSWORD:?set DB_PASSWORD in .env}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U myapi -d myapi"]
      interval: 5s
      retries: 10
    restart: unless-stopped

  cache:
    image: redis:7-alpine
    restart: unless-stopped

volumes:
  pgdata:
```

```
docker compose up -d --build
docker compose ps
docker compose logs -f api
docker compose exec api python manage.py migrate
docker compose exec db psql -U myapi -d myapi
docker compose down          # stop and remove containers (the volume stays)
docker compose down -v       # ALSO delete volumes, including database data
```

> **Note:** the top-level `version:` key used in older examples (including your *Mastery Guide*) is obsolete in Compose v2 and is ignored with a warning; leave it out. Pin image versions (`postgres:17`) rather than using `latest`. Check the image's documentation for the right data path when you change the major version.

17.6 The Docker and UFW trap

Docker writes its own firewall rules for published ports, **bypassing UFW**. `ports: "8000:8000"` makes the API reachable from the Internet even if UFW denies port 8000.

Fixes:

- Publish only on localhost (`127.0.0.1:8000:8000`), as in the Compose file above, and let Nginx proxy to it.
- Don't publish database ports at all. Containers in the same Compose project reach each other by service name (`db:5432`).

17.7 Volumes, backups and clean-up

```
docker volume ls
docker compose exec -T db pg_dump -U myapi -Fc myapi > myapi.dump   # back up from a container
docker image prune -a --filter "until=168h"                         # images unused for 7 days
```

Lab 17 — Containerise the API

1. Write the Dockerfile and `.dockerignore` for your Django project. Keep the image under 250 MB (check with `docker images`).
2. Run the Compose stack. Prove that the database survives `docker compose down` and `up`, but not `down -v`.
3. Show the UFW bypass: publish `8000:8000`, reach it from another machine despite UFW, then fix it with `127.0.0.1:`.
4. Put host Nginx in front of the container with the configuration from 16.6, changing only `proxy_pass` to `http://127.0.0.1:8000`.

Checkpoint 17

1. What is the difference between an image, a container and a volume?
2. Why copy `requirements.txt` and install it before copying the rest of the code?
3. Why is membership of the `docker` group a security concern?
4. How can a published container port bypass UFW, and how do you prevent it?
5. What does `docker compose down -v` destroy?

> **Tutor note:** 6 hours. Sources: Docker Engine installation documentation for Ubuntu, Dockerfile and Compose references, Docker's packet-filtering documentation (UFW interaction); your *Linux Mastery Guide*, Phase 10, modernised (no `version:` key, `npm ci --omit=dev` instead of the deprecated `--production`).

Module 18 — Security hardening

Objectives: apply a repeatable hardening baseline to an Ubuntu server and a Mint workstation: SSH, firewall, brute-force protection, automatic patches, least privilege, secrets handling, mandatory access control and auditing.

> **Mental model:** security is layers of locks (defence in depth). No single setting makes a server safe; each layer catches what the previous one missed.

18.1 Hardening baseline (server)

#	Control	How
1	Patch automatically	`unattended-upgrades` (Module 6); reboot when `/var/run/reboot-required` exists
2	Key-only SSH, no root login	`sshd_config.d` drop-in (18.2)
3	Firewall default-deny	UFW: allow only 22, 80 and 443 (Module 11)
4	Brute-force protection	fail2ban (18.3)
5	Least privilege	One system user per app; no `sudo` for service users; `NOPASSWD` only for specific commands
6	Services bound to localhost	PostgreSQL and Redis on 127.0.0.1; Docker ports on `127.0.0.1:`
7	Secrets out of code	`EnvironmentFile` with mode 640; never in Git or images
8	Sandbox services	systemd `ProtectSystem`, `NoNewPrivileges`, `PrivateTmp`; AppArmor
9	Log and review	`journalctl`, `/var/log/auth.log`, `last`, `lastb`
10	Back up and test restores	Module 10 and 15

18.2 SSH hardening the safe way

Ubuntu reads every file in `/etc/ssh/sshd_config.d/`, so put your settings in a drop-in rather than editing the main file:

```
sudo tee /etc/ssh/sshd_config.d/01-hardening.conf > /dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
AllowUsers amakundi deploy
EOF
sudo sshd -t                      # syntax check: no output means OK
sudo sshd -T | grep -E 'permitrootlogin|passwordauthentication|maxauthtries'
sudo systemctl restart ssh        # the service is "ssh" on Ubuntu and Mint

maxauthtries 3
permitrootlogin no
passwordauthentication no
```

> **Warning:** before you disable passwords, confirm that key login works **in a second terminal**, and keep your current session open until you have tested a fresh login. Cloud providers offer a web console if you lock yourself out, but your own servers may not.
>
> **Warning:** for each setting, `sshd` uses the **first** value it reads, and it reads drop-ins in name order. Cloud images often ship `50-cloud-init.conf` containing `PasswordAuthentication yes`, which would beat a file named `60-…`. That is why the file above is named `01-hardening.conf`. Always confirm the effective values with `sshd -T`.
>
> **Note:** the common recipe `sed -i 's/#PasswordAuthentication yes/…/'` (including in your *Mastery Guide*) silently does nothing if the line looks different. A drop-in file plus `sshd -T` *proves* the effective setting. On Ubuntu 24.04 and later, SSH is socket-activated. If you change `Port`, run `sudo systemctl daemon-reload && sudo systemctl restart ssh.socket`, and allow the new port in UFW *first*.

18.3 fail2ban

```
sudo apt install fail2ban
sudo tee /etc/fail2ban/jail.local > /dev/null <<'EOF'
[DEFAULT]
bantime  = 1h
findtime = 10m
maxretry = 5
backend  = systemd
ignoreip = 127.0.0.1/8 ::1

[sshd]
enabled = true
EOF
sudo fail2ban-client -t            # configuration test
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
sudo fail2ban-client set sshd unbanip <ip>
```

Ubuntu's package already enables the `sshd` jail and uses nftables (`/etc/fail2ban/jail.d/defaults-debian.conf`). `jail.local` overrides the defaults; never edit `jail.conf`, which upgrades replace.

18.4 AppArmor

Ubuntu and Mint ship AppArmor, which confines programs to the files and capabilities in their profile:

```
sudo aa-status | head -20
sudo apt install apparmor-utils
sudo aa-complain /usr/sbin/<program>     # log violations instead of blocking (for testing)
sudo aa-enforce /usr/sbin/<program>
journalctl -k | grep -i apparmor          # denials
```

18.5 Auditing a machine

```
sudo ss -tulpn                        # what listens, and on which address?
sudo ufw status verbose
last -n 10; sudo lastb -n 10          # recent logins / failed logins
sudo journalctl _COMM=sshd --since today | grep -i "failed\|accepted"
find / -perm -4000 -type f 2>/dev/null   # setuid binaries: is anything unexpected?
sudo apt install lynis && sudo lynis audit system   # hardening score and suggestions
```

18.6 Workstation security (Mint)

- **Full-disk encryption:** choose it during installation (LUKS). It is the main protection for a stolen laptop.
- **Screen lock and updates:** enable the screen lock, and let Update Manager install security updates.
- **Firewall:** UFW enabled (Mint's Firewall Configuration tool is a front-end for it).
- **SSH keys:** protect them with a passphrase; use `ssh-agent` rather than passphrase-less keys.
- **Wallets and keys:** keep wallet seed phrases and production keys off the development machine entirely (Module 21).

Lab 18 — Hardening sprint

On a fresh Ubuntu Server VM, apply the baseline from 18.1 as a single idempotent script, `harden.sh`. Then:

1. Show that password SSH login fails and key login works.
2. Trigger a fail2ban ban from another VM with wrong passwords, show the ban, and lift it.
3. Run `lynis audit system` before and after, and record the hardening index.
4. Produce a one-page "server security report" listing every listening port and the reason it is open.

Checkpoint 18

1. Why use a drop-in file in `sshd_config.d` instead of `sed` on `sshd_config`?
2. What must you check before restarting SSH with passwords disabled?
3. What is the difference between `jail.conf` and `jail.local`?
4. Name three services that should normally listen on localhost only.
5. What does AppArmor complain mode do?

> **Tutor note:** 4 hours. Sources: Ubuntu Server security documentation (OpenSSH, firewall, AppArmor), the `sshd_config(5)` man page, fail2ban documentation; your *Linux Mastery Guide*, Phases 9 and 12. Run the lockout scenario on purpose in a VM so learners experience it safely once.

Module 19 — Monitoring, performance and troubleshooting

Objectives: read the four core resources (CPU, memory, disk and network); use the standard observability tools; diagnose common production incidents with a repeatable method; set up lightweight monitoring and alerts.

19.1 The troubleshooting method

1. **Define the problem.** What exactly is wrong, for whom, and since when? What changed (deploy, update, traffic)?
2. **Check the health of the box.** Run `uptime`, `free -h`, `df -h`, `top`.
3. **Check the service.** Run `systemctl status`, then `journalctl -u … -n 100`.
4. **Check the network path.** Run `ss -tulpn`, `curl -v`, and look at the firewall.
5. **Form one hypothesis and test it.** Change one thing at a time.
6. **Fix, verify and write it up.** Write a short post-incident note, and add a check so it can't happen silently again.

> **Mental model:** be a doctor, not a mechanic who replaces random parts. Take the vital signs first (CPU, memory, disk, network), then examine the patient (service), then diagnose.

19.2 The USE method for resources

For each resource, check **U**tilisation, **S**aturation (queued work) and **E**rrors:

Resource	Utilisation	Saturation	Errors
CPU	`top`, `mpstat -P ALL 1`	Load average > cores; `vmstat r` column	`dmesg`
Memory	`free -h` (look at `available`)	Swapping: `vmstat si/so`; OOM kills	`journalctl -k | grep -i oom`
Disk	`iostat -xz 1` (`%util`)	`aqu-sz`, `await`; processes in state `D`	`dmesg`, `smartctl -a /dev/sda`
Network	`ip -s link`, `sar -n DEV 1`	Drops; `ss -s`	`ip -s link` errors

Install the toolkit: `sudo apt install sysstat iotop smartmontools`. `sysstat` also records history; read it with `sar -u` and `sar -r`.

19.3 Reading vmstat

```
vmstat 1 5

procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 1  0      0 812340  90312 1903400    0    0     5    21  310  540  4  1 94  1  0  0
```

The first line shows averages since boot; read the following lines.

Column	What it tells you
`r`	Runnable processes; persistently above the core count means CPU saturation
`b`	Processes blocked on I/O
`si/so`	Swap in/out; any steady value means memory pressure
`us/sy/id/wa`	CPU time spent in user code / kernel / idle / waiting for I/O; high `wa` means slow storage

19.4 Common incidents and their playbooks

Symptom	First commands	Usual causes
Site down / 502	`systemctl status myapi nginx`; `journalctl -u myapi -n 50`	App crash, bad deploy, missing environment variable
Very slow	`top`; `vmstat 1`; database `pg_stat_activity`	CPU-bound workers, slow queries, swapping
"No space left on device"	`df -h`; `df -i`; `du -xh --max-depth=1 /`	Logs, Docker images, old backups, the journal
Process killed suddenly	`journalctl -k | grep -i "out of memory"`	Out-of-memory killer; too many workers
Can't connect	`ss -tulpn`; `ufw status`; `curl -v`	Listening on 127.0.0.1, firewall, DNS
High load, low CPU	`vmstat` (`b`, `wa`); `iotop`	Disk I/O bottleneck
"Too many open files"	`ulimit -n`; `ls /proc/<pid>/fd | wc -l`	Connection or file-handle leak; raise `LimitNOFILE=` in the unit

19.5 Deeper tools

```
sudo strace -f -p <PID> -e trace=network,file   # what system calls a process makes
sudo lsof -p <PID> | head                        # open files and sockets
sudo perf top                                    # CPU hot spots (linux-tools-$(uname -r))
sudo tcpdump -i any port 5432 -c 20              # packets on the wire
dmesg -T | tail -20                              # kernel messages with readable times
```

19.6 Kernel tuning in moderation

```
sysctl vm.swappiness
sudo sysctl -w vm.swappiness=10                                  # now
echo 'vm.swappiness=10' | sudo tee /etc/sysctl.d/90-local.conf # persistent
sudo sysctl --system
```

Change kernel parameters only to fix a measured problem, and record why. Your *Mastery Guide* appends to `/etc/sysctl.conf`; a file in `/etc/sysctl.d/` is cleaner and survives package upgrades.

19.7 Lightweight monitoring

Need	Tool
Is my site up?	An external uptime checker, plus a cron or timer job that `curl`s `/health/` and alerts on failure
Metrics dashboards	Prometheus with `node_exporter` (`sudo apt install prometheus-node-exporter`) and Grafana; or Netdata for a one-command install
Logs in one place	Grafana Loki with Promtail/Alloy, or the ELK stack
Alerts	Grafana or Alertmanager to email, Slack or Telegram

A minimal health-check timer is the first monitoring every service needs: combine `check_port.sh` or `curl -fsS https://…/health/` with the webhook from Module 13.

Lab 19 — Incident drills

Your tutor (or a script) breaks your Module 16 server in each of these ways. Restore service using only the method in 19.1, and write a five-line incident note for each:

1. Fill the disk with `fallocate -l` until it is 100% full.
2. Start a CPU hog: `stress-ng --cpu 4` (`sudo apt install stress-ng`).
3. Set `--workers 20` on a 1 GB VM and send load until the OOM killer acts.
4. Add a UFW rule that denies port 443.
5. Rename the environment file so the app can't start.

Checkpoint 19

1. In `free -h`, why is `available` more meaningful than `free`?
2. What does a high `wa` value in `vmstat` suggest?
3. How do you confirm that the OOM killer terminated your process?
4. Give the six steps of the troubleshooting method.
5. Why should kernel tuning go in `/etc/sysctl.d/` rather than `/etc/sysctl.conf`?

> **Tutor note:** 5 hours. Sources: *The Linux System Administrator's Guide*, chapters 6–7 (memory, monitoring); *Introduction to Linux*, chapter 4; your *Linux Mastery Guide*, Phase 12; Brendan Gregg's USE method. The incident drills are the most valuable exercise in Level 3; run them under time pressure.

Module 20 — CI/CD and cloud servers

Objectives: provision a cloud Ubuntu server safely; automate testing and deployment with GitHub Actions; handle deployment secrets; know when to use containers in the pipeline.

20.1 Provisioning a cloud server: checklist

1. Create an Ubuntu Server LTS VM (24.04 or 26.04) with your **SSH public key** added at creation; never use password login.
2. In the cloud firewall (security group), allow 22 from your IP only if possible, plus 80 and 443 from anywhere.
3. First login: create your user with sudo, apply the Module 18 baseline, enable automatic updates, and set the timezone and hostname.
4. Point a domain's DNS A record at the server's IP.
5. Create a **deploy** user that may run only the deploy script with sudo:

```
sudo adduser --disabled-password --gecos "" deploy
sudo visudo -f /etc/sudoers.d/deploy

deploy ALL=(root) NOPASSWD: /usr/local/bin/deploy-myapi
```

6. Generate a dedicated key pair for CI. Put the public key in `/home/deploy/.ssh/authorized_keys` and the private key in your GitHub repository's secrets.

20.2 GitHub Actions pipeline

```yaml
# .github/workflows/ci-cd.yml
name: CI/CD

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:17
        env:
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: test
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s --health-retries 10
    env:
      DJANGO_SECRET_KEY: ci-only-not-secret
      DATABASE_URL: postgres://postgres:postgres@localhost:5432/test
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -r requirements.txt
      - run: python manage.py check --deploy --fail-level ERROR
      - run: python manage.py test

  deploy:
    needs: test
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest
    environment: production
    steps:
      - name: Deploy over SSH
        env:
          SSH_KEY: ${{ secrets.DEPLOY_SSH_KEY }}
```

```
        HOST: ${{ secrets.DEPLOY_HOST }}
        KNOWN_HOSTS: ${{ secrets.DEPLOY_KNOWN_HOSTS }}
      run: |
        install -m 700 -d ~/.ssh
        printf '%s\n' "$SSH_KEY" > ~/.ssh/id_ed25519 && chmod 600 ~/.ssh/id_ed25519
        printf '%s\n' "$KNOWN_HOSTS" > ~/.ssh/known_hosts
        ssh deploy@"$HOST" 'sudo /usr/local/bin/deploy-myapi'
```

Get `DEPLOY_KNOWN_HOSTS` with `ssh-keyscan -t ed25519 <host>`. Pinning the host key prevents a man-in-the-middle attack on your deployment.

How the pipeline works:

- Every push and pull request runs the tests against a real PostgreSQL service container.
- Only pushes to `main` that pass the tests deploy.
- The deploy key can do exactly one thing on the server: run the deploy script.
- The `production` environment can require a manual approval in the GitHub settings.

20.3 Container-based delivery

When you have adopted Docker (Module 17), the pipeline changes:

1. **Build** the image in CI and tag it with the commit SHA.
2. **Push** it to a registry (GitHub Container Registry: `ghcr.io/<user>/myapi:<sha>`).
3. **Deploy:** on the server, `docker compose pull && docker compose up -d`.
4. **Roll back** by redeploying the previous tag.

20.4 Infrastructure as code (next steps)

When you manage more than two servers, describe them in code:

- **Ansible:** agentless, SSH-based configuration management; turns your `harden.sh` and deploy steps into playbooks.
- **Terraform / OpenTofu:** creates the cloud resources (VMs, DNS, firewalls).
- **cloud-init:** the first-boot configuration that cloud images run automatically.

Lab 20 — Push to deploy

1. Provision a cloud Ubuntu server with the 20.1 checklist.
2. Deploy your project manually once (Module 16).
3. Add the workflow above. Show:
   - a failing test blocks deployment;
   - a passing merge to `main` deploys automatically;
   - the health check passes.
4. Practise rollback: `git revert` the last commit and let the pipeline redeploy.

Checkpoint 20

1. Why does the CI key run only one whitelisted `sudo` command?
2. What does pinning `known_hosts` protect against?
3. Why run tests against a real PostgreSQL in CI rather than SQLite?
4. How do you roll back a bad deploy (a) with Git and (b) with container tags?

> **Tutor note:** 5 hours. Sources: GitHub Actions documentation (services, environments, secrets); your *Linux Mastery Guide*, Phases 11 and 13. Your *Mastery Guide* restarts all processes with `pm2 restart all` over SSH using the server's main account; this manual narrows the deploy key's privileges, which is what security reviews expect.

Level 4 — Web3 engineer

Level 4 applies your Linux and backend skills to blockchain technology. Smart-contract development, testing and deployment tools are built for Unix-like systems, and the nodes that run Ethereum and IPFS are Linux services, so everything from Levels 1–3 transfers directly. Budget about three weeks.

> **Scope:** this level concentrates on the Ethereum/EVM ecosystem, the most widely used smart-contract platform and the one most Web3 backend jobs ask for. The Linux skills (toolchains, key management, nodes as systemd services, firewalls, monitoring) apply equally to other chains.

Module 21 — Web3 development toolchain on Linux

Objectives: install and use Foundry and Hardhat 3; run a local blockchain; compile, test and deploy a contract; interact with chains from the command line and from Python; handle private keys safely.

21.1 How the pieces fit

Piece	Role	Linux angle
Solidity	Smart-contract language	Compiled by `solc`, which the toolchains download for you
Foundry (`forge`, `cast`, `anvil`, `chisel`)	Rust-based toolkit: build, test in Solidity, CLI chain access, local node	Single binaries in `~/.foundry/bin`
Hardhat 3	Node.js-based toolkit: TypeScript/Solidity tests, Ignition deployments, keystore	Needs Node.js ≥ 22.13 (Module 14)
JSON-RPC endpoint	How tools talk to a chain (port 8545 by convention)	`curl`, firewall rules, `localhost` binding
Wallet / private key	Signs transactions	File permissions, encrypted keystores, never in Git
Testnet (Sepolia)	Free test ETH from faucets; behaves like mainnet	Get an RPC URL from a provider or your own node (Module 22)

> **Mental model:** a blockchain node is a public database server that everyone replicates. A smart contract is a stored procedure that anyone can call. A transaction is a signed write request: your private key is the only password, and nobody can reset it.

21.2 Foundry

```
curl -L https://getfoundry.sh/install | bash
source ~/.bashrc          # puts ~/.foundry/bin on PATH
foundryup                 # installs forge, cast, anvil and chisel; rerun it to update
forge --version
```

Create, build and test a project:

```
forge init hello_foundry
cd hello_foundry
forge build
forge test -vv
```

The template contains `src/Counter.sol`, `test/Counter.t.sol` (tests written in Solidity) and `script/Counter.s.sol` (a deployment script).

Run a local chain in a second terminal (or in `tmux`):

```
anvil
```

`anvil` listens on `127.0.0.1:8545` with chain ID 31337 and 10 pre-funded test accounts. It prints their addresses and private keys. Those keys are public knowledge; use them only on local chains.

Talk to the chain with `cast`:

```
export RPC=http://127.0.0.1:8545
cast chain-id --rpc-url $RPC                                    # 31337
cast block-number --rpc-url $RPC
cast balance 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 --ether --rpc-url $RPC
```

Deploy with a script, using an encrypted keystore account rather than a raw key:

```
cast wallet import dev --interactive          # paste an anvil test key; set a password
cast wallet list
forge script script/Counter.s.sol --rpc-url $RPC --account dev --broadcast
```

Without `--broadcast`, `forge script` only simulates the deployment, which is a useful dry run.

Read and write the deployed contract:

```
cast call <counter-address> "number()(uint256)" --rpc-url $RPC
cast send <counter-address> "increment()" --account dev --rpc-url $RPC
```

21.3 Hardhat 3

```
nvm use --lts                    # Node.js >= 22.13 required
mkdir counter-hh && cd counter-hh
npx hardhat --init               # interactive; or non-interactive:
npx hardhat --init --template node-test-runner-viem
npx hardhat --help
```

The generated project:

```
contracts/Counter.sol      contracts/Counter.t.sol (Solidity test)
test/Counter.ts            (TypeScript test with node:test + viem)
ignition/modules/Counter.ts (deployment module)
hardhat.config.ts
```

Main tasks (verified with Hardhat 3.20):

Command	Purpose
`npx hardhat build`	Compile (`compile` is an alias)
`npx hardhat test`	All tests; `test solidity` or `test nodejs` for one kind
`npx hardhat node`	Local JSON-RPC node on `127.0.0.1:8545`, chain ID 31337
`npx hardhat ignition deploy ignition/modules/Counter.ts`	Deploy to an in-process simulated network
`npx hardhat ignition deploy ignition/modules/Counter.ts --network sepolia`	Deploy to Sepolia
`npx hardhat keystore set SEPOLIA_PRIVATE_KEY`	Store a secret encrypted in Hardhat's keystore

`npx hardhat node` prints pre-funded accounts, just like `anvil`:

```
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
========

WARNING: Funds sent on live network to accounts with publicly known private keys WILL BE LOST.

Account #0:  0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266 (10000 ETH)
```

The Sepolia network in the generated `hardhat.config.ts` reads its secrets through `configVariable`, never from the code:

```
sepolia: {
  type: "http",
  chainType: "l1",
  url: configVariable("SEPOLIA_RPC_URL"),
  accounts: [configVariable("SEPOLIA_PRIVATE_KEY")],
},

npx hardhat keystore set SEPOLIA_RPC_URL
npx hardhat keystore set SEPOLIA_PRIVATE_KEY
npx hardhat ignition deploy ignition/modules/Counter.ts --network sepolia
```

Ignition records each deployment in `ignition/deployments/chain-<id>/`. Commit that directory so your team knows what is deployed where.

> **Note:** if `build` fails with `HHE905: Couldn't download compiler version list`, the machine cannot reach the Solidity compiler download server. Check the network, proxy and DNS (Module 11); firewalled corporate or university networks often cause this.

21.4 Querying a chain from the terminal

Every node speaks JSON-RPC, so `curl` and `jq` (Module 7) are enough to inspect one:

```
curl -s -X POST -H 'Content-Type: application/json' \
  --data '{"jsonrpc":"2.0","method":"eth_chainId","params":[],"id":1}' \
  http://127.0.0.1:8545 | jq

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": "0x7a69"
}
```

`0x7a69` is 31337 in hexadecimal. Convert it with `printf '%d\n' 0x7a69` or `cast to-dec 0x7a69`.

21.5 A Python block watcher (backend meets Web3)

Backends often watch chains: indexing events, updating a database and notifying users. This `web3.py` script is the skeleton of such a service. It was tested against a local Hardhat node.

File: `watch_blocks.py`

```
"""watch_blocks.py - print new blocks from an Ethereum JSON-RPC endpoint."""
import os
import time

from web3 import Web3

RPC_URL = os.environ.get("RPC_URL", "http://127.0.0.1:8545")
w3 = Web3(Web3.HTTPProvider(RPC_URL))

if not w3.is_connected():
    raise SystemExit(f"cannot reach {RPC_URL}")

print(f"connected: chain_id={w3.eth.chain_id} block={w3.eth.block_number}")
last = w3.eth.block_number
while True:
    current = w3.eth.block_number
    for n in range(last + 1, current + 1):
        block = w3.eth.get_block(n)
        print(f"block {n}: {len(block.transactions)} tx, gas used {block.gasUsed}", flush=True)
    last = current
    time.sleep(2)
```

```
python3 -m venv .venv && source .venv/bin/activate
pip install web3
python watch_blocks.py

connected: chain_id=31337 block=0
block 1: 1 tx, gas used 21000
```

To run it permanently, give it its own system user and a systemd unit, exactly as in Module 16, with `Restart=always` and `Environment=RPC_URL=…`. In production you would subscribe to contract events, store progress in PostgreSQL so a restart doesn't miss blocks, and handle chain reorganisations by waiting a few confirmations.

21.6 Key and secret hygiene

Rule	How on Linux
Separate keys by purpose	A local test key (anvil/Hardhat), a testnet deployer, and production keys on a hardware wallet or multisig
Never store raw keys in plain files or Git	Use `cast wallet import` (encrypted keystore in `~/.foundry/keystores`, mode 600) or `npx hardhat keystore set`; add `.env` to `.gitignore`
Keep secrets out of shell history	Use the interactive prompts. A command line containing a key is saved in `~/.bash_history`; start such a

Rule	How on Linux
	command with a space when `HISTCONTROL=ignorespace` is set (the Ubuntu default)
Scan before pushing	`gitleaks detect`
Assume a leaked key is lost	Move the funds immediately and rotate the key; on-chain theft can't be reversed

> **Warning:** never enter a seed phrase or production private key on a development machine that runs untrusted code: `npm install` scripts, unknown repositories, or "test this contract for me" projects from strangers. Malicious packages that steal wallet files are a documented, ongoing attack against Web3 developers.

Lab 21 — From zero to deployed

1. Install Foundry and Hardhat 3 on your Mint workstation.
2. In Foundry, extend `Counter` with a `decrement()` function that reverts at zero. Write tests for both the normal path and the revert, and reach 100% line coverage (`forge coverage`).
3. Run `anvil` in `tmux`, deploy with `forge script --account dev --broadcast`, and change the value with `cast send`.
4. In Hardhat, deploy the template's Counter with Ignition to the simulated network, then to `npx hardhat node`.
5. Run `watch_blocks.py` as a systemd **user** service (`systemctl --user`) and watch it log your `cast send` transactions.
6. **Optional:** get Sepolia test ETH from a faucet and deploy to Sepolia using the keystore. Record the transaction hash and view it in a block explorer.

Checkpoint 21

1. What do `forge`, `cast` and `anvil` each do?
2. Why use `cast wallet import` or `hardhat keystore` instead of a `PRIVATE_KEY=` line in `.env`?
3. Which Node.js version does Hardhat 3 need, and how do you make sure the project uses it?
4. What does `forge script` do without `--broadcast`?
5. Why must a production chain-watcher store the last processed block somewhere durable?

> **Tutor note:** 6 hours. Sources: Foundry documentation (installation, getting started, deploying), Hardhat 3 documentation (getting started, Ignition, keystore), `web3.py` documentation. The Hardhat commands and outputs in this module were run on Linux while preparing the manual. Emphasise 21.6 as much as the tooling: key handling is where real money is lost.

Module 22 — Running blockchain infrastructure on Linux

Objectives: plan the hardware and network for an Ethereum node; install an execution client and a consensus client; run both as hardened systemd services; secure the RPC; monitor sync; run an IPFS (Kubo) node as a service.

22.1 What an Ethereum node is (since the Merge)

A full Ethereum node is **two programs** that talk over an authenticated local API (the Engine API, port 8551, secured by a shared JWT secret):

Layer	Clients (choose one of each)	Responsibilities	P2P port
Execution	Geth, Nethermind, Besu, Erigon, Reth	Transactions, EVM, state, JSON-RPC (8545)	30303 TCP/UDP
Consensus	Lighthouse, Prysm, Teku, Nimbus, Lodestar	Proof-of-stake, blocks and attestations, beacon API (5052)	9000 TCP/UDP (Lighthouse)

Choosing less common clients ("client diversity") makes the network more resilient.

Plan the resources before you start:

Network	Use	Rough requirements (check each client's docs; they grow over time)
Sepolia	App testing	4 cores, 16 GB RAM, ~1 TB SSD
Hoodi	Staking and validator testing	Similar to Sepolia
Mainnet	Production	4–8 cores, 32 GB RAM, 2–4 TB **NVMe** SSD, unmetered broadband

> **Note:** for development, use `anvil` or a hosted RPC provider. Run your own node when you need privacy, reliability without rate limits, archive data or a validator. That is a real infrastructure job and a strong portfolio item.

22.2 Prepare the server

```
sudo apt update && sudo apt full-upgrade -y
sudo adduser --system --group --home /var/lib/geth --shell /usr/sbin/nologin geth
sudo adduser --system --group --home /var/lib/lighthouse --shell /usr/sbin/nologin lighthouse
sudo mkdir -p /var/lib/ethereum-secrets
openssl rand -hex 32 | tr -d '\n' | sudo tee /var/lib/ethereum-secrets/jwt.hex > /dev/null
sudo groupadd ethjwt
sudo usermod -aG ethjwt geth && sudo usermod -aG ethjwt lighthouse
sudo chown root:ethjwt /var/lib/ethereum-secrets/jwt.hex
sudo chmod 640 /var/lib/ethereum-secrets/jwt.hex
sudo ufw allow 30303 comment 'eth execution p2p'
sudo ufw allow 9000 comment 'eth consensus p2p'
```

Put the data directories on the fast NVMe disk. If it is a separate disk, mount it with Module 10's fstab method before you start.

22.3 Install the clients

Geth from its official PPA (Ubuntu; this also works on Mint because PPAs follow the Ubuntu base):

```
sudo add-apt-repository -y ppa:ethereum/ethereum
sudo apt update && sudo apt install -y ethereum
geth version
```

Lighthouse from its release binaries:

1. Download the `x86_64-unknown-linux-gnu` tarball for the latest release from github.com/sigp/lighthouse/releases.
2. Verify its checksum and signature as the release notes describe.
3. Install it:

```
tar -xzf lighthouse-*-x86_64-unknown-linux-gnu.tar.gz
sudo install -m 755 lighthouse /usr/local/bin/
lighthouse --version
```

22.4 systemd units

```
# /etc/systemd/system/geth.service
[Unit]
Description=Geth execution client (Sepolia)
After=network-online.target
Wants=network-online.target

[Service]
User=geth
Group=geth
ExecStart=/usr/bin/geth --sepolia \
    --datadir /var/lib/geth \
    --authrpc.addr 127.0.0.1 --authrpc.port 8551 \
    --authrpc.jwtsecret /var/lib/ethereum-secrets/jwt.hex \
    --http --http.addr 127.0.0.1 --http.port 8545 --http.api eth,net,web3 \
    --metrics --metrics.addr 127.0.0.1
Restart=on-failure
RestartSec=10
TimeoutStopSec=300
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/var/lib/geth

[Install]
WantedBy=multi-user.target

# /etc/systemd/system/lighthouse-bn.service
[Unit]
Description=Lighthouse beacon node (Sepolia)
After=network-online.target geth.service
Wants=network-online.target

[Service]
User=lighthouse
Group=lighthouse
ExecStart=/usr/local/bin/lighthouse bn --network sepolia \
    --datadir /var/lib/lighthouse \
    --execution-endpoint http://127.0.0.1:8551 \
    --execution-jwt /var/lib/ethereum-secrets/jwt.hex \
    --checkpoint-sync-url <checkpoint-sync-url> \
    --http --http-address 127.0.0.1 --http-port 5052
Restart=on-failure
RestartSec=10
TimeoutStopSec=300
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/var/lib/lighthouse

[Install]
WantedBy=multi-user.target
```

Choose a checkpoint-sync URL for your network from the community list of checkpoint-sync endpoints linked from the Lighthouse documentation. Checkpoint sync makes the beacon node usable in minutes instead of days.

```
sudo systemctl daemon-reload
sudo systemctl enable --now geth lighthouse-bn
journalctl -fu geth
journalctl -fu lighthouse-bn
```

Unit setting	Why it is there
`TimeoutStopSec=300`	Clients need time to flush their databases on shutdown;

Unit setting	Why it is there
	killing them early can corrupt data
RPC and auth ports on `127.0.0.1` only	Never expose 8545 or 8551 publicly. For remote access, use an SSH tunnel or an authenticated Nginx proxy (Module 16) with IP allow-lists
`--http.api eth,net,web3`	Don't expose `admin`, `debug` or `personal`

22.5 Checking sync and health

```
geth attach --exec 'eth.syncing' /var/lib/geth/geth.ipc
curl -s -X POST -H 'Content-Type: application/json' \
  --data '{"jsonrpc":"2.0","method":"eth_syncing","params":[],"id":1}' http://127.0.0.1:8545 | jq
curl -s http://127.0.0.1:5052/eth/v1/node/syncing | jq
curl -s http://127.0.0.1:5052/eth/v1/node/peer_count | jq
df -h /var/lib/geth
```

`eth_syncing` returns `false` when the node is synced.

Monitoring builds on Module 19:

- Prometheus can scrape Geth's metrics (`--metrics`) and Lighthouse's (add `--metrics`), with node exporter for the host.
- The clients publish Grafana dashboards.
- Alert on peer count, sync status, disk usage over 80%, and service restarts.

Upgrades:

- Geth updates with `apt`.
- Lighthouse updates by replacing the binary and restarting.
- Read the release notes first: network upgrades (hard forks) have **deadlines**. A client that is not updated in time follows the wrong chain.

22.6 IPFS with Kubo

IPFS stores dApp front-ends and NFT metadata by content address. Run your own node:

1. Download the `kubo` Linux tarball from dist.ipfs.tech, verify its checksum, then install it:

```
tar -xzf kubo_*_linux-amd64.tar.gz
sudo bash kubo/install.sh
ipfs --version
```

2. Create a service user and initialise the repository with the server profile, which avoids scanning local networks:

```
sudo adduser --system --group --home /var/lib/ipfs --shell /usr/sbin/nologin ipfs
sudo -u ipfs IPFS_PATH=/var/lib/ipfs/.ipfs ipfs init --profile server
```

3. Create the service unit:

```
# /etc/systemd/system/ipfs.service
[Unit]
Description=IPFS Kubo daemon
After=network-online.target
Wants=network-online.target

[Service]
User=ipfs
Environment=IPFS_PATH=/var/lib/ipfs/.ipfs
ExecStart=/usr/local/bin/ipfs daemon --enable-gc
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

4. Open the swarm port, start the service, and add a file:

```
sudo ufw allow 4001 comment 'ipfs swarm'
sudo systemctl daemon-reload && sudo systemctl enable --now ipfs
sudo -u ipfs IPFS_PATH=/var/lib/ipfs/.ipfs ipfs add -r ./dist      # publish a built front-end
```

The API (5001) and gateway (8080) listen on localhost by default. Keep the API private: it controls the node.

Lab 22 — Node operator

1. On a VM or VPS that meets the Sepolia requirements, install Geth and Lighthouse as hardened services, following 22.2–22.4.
2. Reach sync. Document how long checkpoint sync took, peer counts, and disk usage.
3. Point your Module 21 block watcher (`RPC_URL=http://127.0.0.1:8545`) at your own node.
4. Prove that ports 8545 and 8551 are **not** reachable from another machine, and that 30303 and 9000 are.
5. Add Prometheus node exporter metrics, and an alert script for disk usage over 80% and for `eth_syncing` not being `false`.
6. Run Kubo, publish a static site built in Module 16, and fetch it through a public gateway by its CID.

Checkpoint 22

1. Why does a post-Merge Ethereum node need two clients, and how do they authenticate to each other?
2. Which ports must be public, and which must never be?
3. Why is `TimeoutStopSec` raised for blockchain clients?
4. What is checkpoint sync, and why use it?
5. What happens to a node that misses a hard-fork upgrade?

> **Tutor note:** 6 hours plus waiting time for synchronisation. Sources: ethereum.org “Run a node” documentation, the Geth and Lighthouse books, Kubo documentation. If hardware is limited, run the lab on Sepolia with a cloud VM for a week, then destroy it. Client flags change between releases, so have learners check `geth --help` and `lighthouse bn --help` against the units above; reading upstream documentation is part of the skill.

Capstone projects

Each capstone is a portfolio piece. Put the code, scripts and a README with architecture notes and screenshots in a public GitHub repository, and leave out every secret.

Capstone 1 — Professional Linux Mint workstation (Levels 1–2)

Deliverable: a `dotfiles` repository with a `bootstrap.sh` that turns a fresh Mint install into your development machine.

The script must:

- install your APT, Flatpak and developer tools;
- configure Git, SSH, aliases and `~/bin`;
- set up Timeshift and a `restic` backup timer for `~/projects`;
- enable UFW;
- be idempotent and pass ShellCheck.

Evidence: a screen recording of the script run on a fresh VM snapshot, and a restore test.

Capstone 2 — Production Django API (Level 3)

Deliverable: your Django REST API, running on an Ubuntu cloud server.

Area	Requirement
Runtime	Gunicorn under systemd, dedicated user, sandboxing options
Edge	Nginx reverse proxy, HTTPS (Let's Encrypt), HTTP→HTTPS redirect
Data	PostgreSQL bound to localhost, nightly `pg_dump` timer with off-site copy, tested restore
Security	Key-only SSH, UFW, fail2ban, unattended-upgrades, `lynis` report
Delivery	GitHub Actions: tests on every pull request, auto-deploy on `main`, health check, rollback documented
Operations	Health-check timer with a webhook alert; a runbook covering at least five incident playbooks

Evidence: the URL, the SSL Labs grade, the CI history, and the runbook.

Capstone 3 — Containerised platform (Level 3)

Deliverable: the same API as a Compose stack (API, PostgreSQL, Redis and a worker such as Celery). The image is built in CI and pushed to GHCR, and the server pulls and runs it with ports bound to localhost behind host Nginx.

Evidence: image size, a demonstration of the UFW bypass and its fix, and backup/restore of the database volume.

Capstone 4 — Web3 backend and node (Level 4)

Deliverable:

1. A tested smart contract, built with Foundry or Hardhat 3, deployed to Sepolia with an encrypted keystore.
2. A Python indexer service (systemd) that reads the contract's events from **your own** Sepolia node, stores them in PostgreSQL, and serves them through a Django REST endpoint behind Nginx.

3. Monitoring that alerts on node sync loss, indexer lag, and disk over 80%.

Evidence: the contract address on a block explorer, the API URL, the Grafana screenshots, and an architecture diagram.

Appendix A — Command cheat sheet

Task	Command
Where am I / who am I	`pwd`, `whoami`, `id`
List with details / hidden files	`ls -lah`
Find files by name / age / size	`find / -name '*.conf' 2>/dev/null`, `-mtime -1`, `-size +100M`
Search inside files	`grep -rniE 'pattern' dir/`
View / follow logs	`less file`, `tail -F file`, `journalctl -fu svc`
Disk and memory	`df -hT`, `du -xh --max-depth=1 / | sort -h`, `free -h`
Processes	`ps aux --sort=-%cpu | head`, `top`/`htop`, `pgrep -a name`
Stop a process	`kill PID` (TERM), `kill -9 PID` (last resort), `pkill -f pattern`
Services	`systemctl status|start|stop|restart|reload|enable --now svc`
Failed services / timers	`systemctl --failed`, `systemctl list-timers`
Network addresses / routes	`ip -br a`, `ip r`
Listening ports	`sudo ss -tulpn`, `sudo lsof -i :PORT`
DNS / HTTP test	`dig +short host`, `curl -sSI https://host`
Remote work	`ssh host`, `scp f host:/p`, `rsync -avz src/ host:dst/`, `ssh -L 5433:localhost:5432 host`
Firewall	`sudo ufw status numbered`, `sudo ufw allow 443/tcp`
Packages	`sudo apt update && sudo apt full-upgrade`, `apt policy pkg`, `dpkg -S /path`
Permissions	`chmod 640 f`, `chmod u+x f`, `sudo chown user:group f`, `namei -l /path`
Archives	`tar -czf a.tgz dir/`, `tar -xzf a.tgz -C dest/`, `tar -tf a.tgz`
Scheduling	`crontab -e`, systemd `.timer` + `systemd-analyze calendar`
Containers	`docker compose up -d`, `docker compose logs -f`, `docker system df`
Web3	`anvil`, `forge test`, `cast block-number --rpc-url $RPC`, `npx hardhat test`

Appendix B — Quick references

Permission values

Octal	Symbolic	Use
600	`rw-------`	Private keys, `.env`, `~/.ssh/config`
640	`rw-r-----`	Service environment files (`root:appgroup`)
644	`rw-r--r--`	Normal files, static web files
700	`rwx------`	`~/.ssh`, private directories
750	`rwxr-x---`	Application directories
755	`rwxr-xr-x`	Scripts, program and web directories
2775	`rwxrwsr-x`	Shared team directories (setgid)

Cron schedules

Expression	Runs
`*/5 * * * *`	Every 5 minutes
`0 * * * *`	Hourly
`30 2 * * *`	Daily at 02:30
`0 9 * * 1-5`	Weekdays at 09:00
`0 0 * * 0`	Sundays at midnight
`0 3 1 * *`	1st of the month at 03:00

The systemd equivalent of `30 2 * * *` is `OnCalendar=*-*-* 02:30:00`.

Common ports

Port	Service	Port	Service
22	SSH	5432	PostgreSQL
25/587	SMTP	6379	Redis
53	DNS	8000	Django/Gunicorn (development)
80 / 443	HTTP / HTTPS	8545 / 8551	Ethereum JSON-RPC / Engine API
3306	MySQL/MariaDB	30303 / 9000	Ethereum P2P (execution / Lighthouse)
3000	Node.js (development)	4001 / 5001	IPFS swarm / API

Signals: `1` HUP (reload), `2` INT (Ctrl+C), `9` KILL, `15` TERM (default), `19` STOP, `18` CONT.

Appendix C — Interview question bank

These are the questions junior backend, DevOps and Web3 infrastructure interviews actually ask. Practise answering aloud in under two minutes each.

Foundations

1. What is the difference between the kernel, the shell and a distribution?
2. Explain `/etc`, `/var`, `/usr`, `/home` and `/proc`.
3. What does `chmod 750` do? When do you use 600?
4. What happens when you type `ls -l` and press Enter (shell parsing, PATH lookup, fork/exec, system calls)?
5. Hard link versus symbolic link?
6. How do you install, remove and find the owner of a file's package on Ubuntu?

Power user

7. Find the 10 IPs with the most requests in an Nginx log.
8. Explain stdin, stdout and stderr, and redirect all output of a command to a file.
9. SIGTERM versus SIGKILL? What is a zombie process?
10. A service won't start. Walk me through your steps.
11. What is the load average and how do you interpret it?
12. The disk is full. How do you find the cause? What if `df` shows free space but writes fail?
13. Explain what `set -euo pipefail` does.
14. Cron versus systemd timers?
15. How do you make a mount permanent, and why use UUIDs?

Networking and security

16. Explain DNS resolution from `curl https://example.com` to the TCP connection.
17. The app works on the server with `curl localhost` but not from outside. Why?
18. How does SSH key authentication work? How do you harden SSH?
19. What does `ssh -L` do? Give a real use.
20. How can Docker bypass UFW?
21. List your hardening baseline for a new Ubuntu server.

Backend operations

22. Deploy a Django app to a fresh Ubuntu server. Describe the architecture and each step.
23. What causes a 502 from Nginx?
24. Why run the app as a non-root user with a `nologin` shell?
25. How do you manage secrets in production?
26. How do you back up PostgreSQL, and how do you know the backup works?
27. Explain a CI/CD pipeline you built.
28. Image versus container versus volume? Why multi-stage or slim images?

Web3 infrastructure

29. Why does an Ethereum node run two clients? Which ports are public?
30. How do you store deployer keys safely on Linux?
31. How would you build a reliable service that indexes contract events?
32. What happens if you don't upgrade a node before a hard fork?

Appendix D — Checkpoint answer key (for tutors)

Answers are short; expect learners to explain them in their own words.

Module 1.

1. The kernel manages hardware and processes; a distribution bundles the kernel with tools, a package manager, services and a desktop.
2. Mint uses Ubuntu LTS packages, so the commands and packages match Ubuntu servers.
3. `UBUNTU_CODENAME`, because `VERSION_CODENAME` is Mint's own name, which third-party repositories don't recognise.
4. `update` refreshes the package lists; `full-upgrade` installs newer versions.
5. Timeshift protects system files, not personal data in `/home` by default.

Module 2.

1. `#` means you are root.
2. They are the same: short and long forms of the same options.
3. `man -k keyword` (`apropos`).
4. `~/.bashrc`; apply with `source ~/.bashrc`.

Module 3.

1. `/etc` holds configuration; `/var/log` holds logs.
2. `../config/dev.env`.
3. `rmdir` deletes only empty directories; `rm -r` deletes recursively.
4. Before. The command receives the expanded list, and an unmatched pattern is passed literally.
5. A hard link is the same inode, can't cross filesystems and can't point to a directory; a symlink is a path pointer that can dangle.

Module 4.

1. `tail -F`.
2. `Esc` then `:q!`.
3. It is present on almost every server, including minimal images.

Module 5.

1. `rw-r-----`: config readable by a service's group but not by others.
2. Deleting changes the directory's entries.
3. setgid: new files inherit the directory's group.
4. It replaces all supplementary groups, so you can lose `sudo`.
5. `visudo` validates the syntax; a broken sudoers file blocks `sudo`.
6. Any compromise of the app would then give full control of the system.

Module 6.

1. `purge` also removes system configuration files.
2. `Signed-By` limits trust in the key to one repository.
3. They use `VERSION_CODENAME`; use `UBUNTU_CODENAME` instead.
4. For newer desktop apps or sandboxing.
5. Boot an older kernel or restore Timeshift; `dpkg --configure -a`; `apt -f install`; downgrade with `pkg=version`.

Module 7.

1. Redirections are processed left to right, so `2>&1` must point stderr to wherever stdout already goes.
2. The redirection happens in your unprivileged shell; use `| sudo tee`.

3. `uniq` only collapses *adjacent* duplicates.
4. `grep -E '^(ERROR|WARN)'`.
5. For variable whitespace, conditions, calculations or reordering fields.

Module 8.

1. On 2 cores it means overloaded (twice the capacity); on 8 cores it means about 50% busy.
2. TERM can be caught for a clean shutdown; KILL cannot.
3. Uninterruptible sleep, usually waiting on I/O.
4. No supervision, no restart, no logs and no boot start.
5. `sudo ss -tulpn | grep 5432` or `sudo lsof -i :5432`.

Module 9.

1. `enable` means start at boot; `start` means start now.
2. In `/etc/systemd/system`, because package upgrades overwrite `/usr/lib`.
3. `journalctl -b -1 -p err`.
4. Any two of: journal logging, catch-up with `Persistent=`, dependencies, calendar validation.
5. It reloads the unit files; needed after creating or editing one.

Module 10.

1. Device names can change between boots.
2. Inode exhaustion.
3. Three copies, two media, one off-site.
4. It deletes files at the destination that aren't in the source; a wrong path can wipe data.
5. Physical volumes, volume groups and logical volumes.

Module 11.

1. Which address it listens on (`ss`), the host firewall (UFW), the cloud firewall or DNS.
2. `127.0.0.1` is local-only; `0.0.0.0` means all interfaces.
3. `netplan try` rolls back automatically if you lose access.
4. It forwards local port 5433 through SSH to port 5432 on the remote host's localhost.
5. `ss -tulpn`.

Module 12.

1. Single quotes are literal; double quotes expand the variable.
2. `"$@"` keeps each argument separate.
3. `/tmp/a/b` and `b.txt`.
4. A script runs in a child process; `source` runs in the current shell.
5. It splits on whitespace and globs; use `while IFS= read -r line; do …; done < file`.

Module 13.

1. `-E`: ERR traps also fire in functions; `-e`: exit on error; `-u`: unset variables are errors; `pipefail`: any failing stage fails the pipeline.
2. A unique name avoids collisions and races, and the trap guarantees cleanup.
3. It prevents overlapping runs.
4. A pipe runs the loop in a subshell; process substitution keeps the loop in the current shell.
5. Any three of: journal logging, `Persistent=`, dependencies, resource controls, easier monitoring.

Module 14.

1. PEP 668 marks the system Python as externally managed, which protects OS tools.
2. `pipx` installs an isolated command-line application; `pip` in a venv installs a project dependency.
3. It pins the project's Node.js version for `nvm use`.
4. Revoke and rotate the key immediately, check for misuse, and clean the history afterwards.

Module 15.

1. Peer authentication maps the Linux user to the database role of the same name, so the local `postgres` user logs in without a password.
2. `postgresql.conf` (`listen_addresses`) and `pg_hba.conf`.
3. It is compressed, supports selective and parallel restore, and can be inspected.
4. Bind to localhost, set authentication if anything remote connects, and set `maxmemory` with an eviction policy.

Module 16.

1. Nginx handles TLS, static files, buffering of slow clients and limits, and protects the app server.
2. Least privilege: no interactive login and no root.
3. The upstream (Gunicorn) is down or unreachable; check the service status, socket permissions and the logs.
4. Secrets stay out of Git, have restricted permissions and can be rotated without code changes.
5. `reload` sends HUP for a graceful worker restart without dropping connections.

Module 17.

1. An image is a template, a container is a running instance, and a volume is persistent data.
2. Layer caching: dependencies are reinstalled only when `requirements.txt` changes.
3. The group is root-equivalent.
4. Docker inserts its own firewall rules for published ports; publish on `127.0.0.1` or not at all.
5. Containers, networks *and* named volumes, which means the data.

Module 18.

1. It survives upgrades, is explicit, and `sshd -T` verifies the result.
2. That key login works in a second session.
3. `jail.local` overrides `jail.conf`, which upgrades replace.
4. PostgreSQL, Redis, and an Ethereum RPC (or a Docker database).
5. It logs violations instead of blocking them.

Module 19.

1. `available` estimates the memory usable without swapping, because it counts reclaimable cache.
2. Processes are waiting on disk I/O, so storage is the bottleneck.
3. `journalctl -k | grep -i 'out of memory'`.
4. Define the problem; check the box; check the service; check the network; hypothesise and test; fix and document.
5. Files in `sysctl.d/` are modular, easy to audit and safe from package changes.

Module 20.

1. Least privilege: a leaked CI key can only run the deploy.
2. A man-in-the-middle impersonating the server.
3. To match production behaviour (SQL dialect, constraints, types).
4.
    (a) `git revert`, then the pipeline redeploys; (b) redeploy the previous image tag.

Module 21.

1. `forge` builds, tests and deploys; `cast` is a command-line client for chains; `anvil` is a local node.
2. The key is encrypted at rest, isn't left in plain files or history, and needs a password.
3. Node.js 22.13 or later; use `nvm` with `.nvmrc`.
4. It simulates the deployment without sending transactions.
5. So it can resume after a restart without missing or duplicating events.

Module 22.

1. After the Merge, execution and consensus are separate clients; they authenticate over the Engine API with a shared JWT.
2. P2P ports (30303, 9000) are public; RPC and auth ports (8545, 8551, 5052) must never be.
3. They need time to flush their databases on shutdown.
4. Checkpoint sync starts from a recent finalised state, so the beacon node syncs in minutes.
5. It follows the old rules, splits from the canonical chain and serves wrong data.

Appendix E — How the source documents were used

Source (supplied)	Year	Used in	What is still valid	What was updated
Introduction to Linux — A Hands-on Guide (two identical copies)	2008	Modules 1–5, 8, 10, 11	Filesystem concepts, permissions, processes, I/O redirection, backup principles	Desktop and print chapters omitted; `ifconfig` → `ip`; SysV → systemd
Bash Guide for Beginners	2009	Modules 7, 12, 13	Regular expressions, `sed`, `awk`, conditionals, loops, functions, signals and traps	`[[ ]]` preferred over `[ ]`; strict mode; ShellCheck; `getopts` patterns
Linux Command Line and Shell Scripting Bible (3rd ed.)	2015	Modules 2–7, 9, 12, 13	Shell, environment variables, permissions, filesystems, packages, editors, scripting, `sed`/`gawk`	Package sources (deb822, keyrings); Snap/Flatpak; modern Ubuntu releases
The Linux System Administrator's Guide	2005	Modules 9, 10, 19	Disks, filesystems, memory, monitoring, accounts, backup strategy	`init`/LILO → systemd/GRUB 2; tapes and floppies → `rsync`/`restic`; journald
The Linux Network Administrator's Guide (2nd ed.)	2001	Modules 11, 18	TCP/IP, routing, DNS concepts, firewall and NAT concepts	`ifconfig`/`route`/`ipchains` → `ip`/`ss`/nftables/UFW; PPP, SLIP, UUCP, NIS and news omitted
Linux Mastery Guide (DOCX and PDF)	2026	All of Levels 1–3; capstones; interview questions	Roadmap structure, phases, mini-projects, interview themes	Ubuntu 22.04 → 24.04/26.04 and Mint 22.3; nvm v0.39 → v0.40.x; Compose `version:` removed; `npm ci --production` → `--omit=dev`; safer SSH hardening, backup rotation and sysctl practice; Web3 levels added

The official documentation consulted includes:

- man pages;
- the GNU Bash Reference Manual;
- systemd man pages;
- Ubuntu Server documentation;
- the Linux Mint user guide and release announcements;
- Docker Engine documentation for Ubuntu;

- Nginx, Gunicorn, Django, PostgreSQL and Certbot documentation;
- Foundry and Hardhat 3 documentation;
- ethereum.org node documentation;
- the Geth and Lighthouse books;
- IPFS Kubo documentation.

Appendix F — Suggested 20-week schedule

The schedule assumes about 8 hours a week. Full-time learners can halve it.

Weeks	Modules	Milestone
1	1–2	Mint and Ubuntu VMs running; comfortable in the terminal
2–3	3–4	File management and editors without the mouse
4	5–6	Users, permissions and software management
5–6	7	Log analysis with pipelines
7	8–9	Processes, services, journal and timers
8	10	Storage and backups with tested restores
9	11	Networking, SSH and firewall
10–11	12–13	Scripting; Capstone 1
12	14–15	Developer workstation; PostgreSQL and Redis
13–14	16	First production deployment with HTTPS
15	17	Docker and Compose
16	18–19	Hardening and incident drills
17	20	CI/CD; Capstone 2
18	21	Web3 toolchain
19	22	Running nodes
20	—	Capstones 3–4, portfolio and interview practice

Certifications to consider afterwards: LPI Linux Essentials (entry level), CompTIA Linux+, and the Linux Foundation's LFCS (hands-on, performance-based). This manual covers most of the practical skills those exams test; check each exam's current objectives before booking.

Appendix G — Glossary

Term	Meaning
Daemon	A background service process (often ending in `d`: `sshd`, `systemd`)
Distribution	Kernel plus tools, packages and defaults (Mint, Ubuntu)
Environment variable	A key=value setting inherited by child processes
FHS	Filesystem Hierarchy Standard: where files belong
Inode	The filesystem record holding a file's metadata
JWT secret	Shared key authenticating an Ethereum execution client and consensus client
LTS	Long-Term Support release (Ubuntu: 5 years of standard support)
Mount	Attaching a filesystem at a directory in the tree
Package	Installable bundle of software plus metadata (`.deb`)
PID	Process ID
Reverse proxy	Server (Nginx) that receives client requests and forwards them to app servers
RPC endpoint	HTTP/WebSocket interface through which tools talk to a blockchain node
Shebang	The `#!` first line that names a script's interpreter
Signal	An asynchronous notification sent to a process (TERM, KILL, HUP)
Socket	An endpoint for communication: TCP port or unix socket file
sudo	Run a command with another user's (usually root's) privileges, with logging
Systemd unit	A file describing a service, timer, mount or target
UFW	Uncomplicated Firewall, a front-end to nftables
Virtual environment	An isolated Python package set per project